Frames All the Way Down

The Missing Foundation

Joshua Chambers
April 2026

Abstract

Meaning is absent from every infrastructure layer of the modern computing stack. It lives exclusively in application code, making that the only route from bytes to anything a user cares about. Applications themselves have migrated off users' devices and onto corporate server farms: the code runs on infrastructure users do not own or control, and the most capable consumer hardware ever built serve as little more than rendering terminals. Users cannot leave because their data is meaningless without the application, which is not something they run.

This paper argues that both gaps — the absence of meaning from the infrastructure and the absence of computation from the user's device — must be closed in the same layer. It proposes a base layer built on two co-equal pillars: a shared semantic commons and a local-first peer-to-peer substrate. The semantic primitive is the *frame*, a predicate-role structure from computational linguistics where keys are grounded meanings rather than strings. The substrate is a peer network of local runtimes holding content-addressed, cryptographically signed assertions. It builds on the foundations of public-key cryptography's promise of identity without a central authority, and PGP's web of trust for establishing that identity peer-to-peer. Both are built into the architecture itself, where trust drives not just key verification but routing, moderation, code execution, discovery, and replication.

In the resulting architecture, applications no longer own the data they operate on, and switching between them does not mean losing your work. Moderation emerges from a trust matrix rather than corporate policy. Routing privacy, from direct connection to multi-hop onion routing, becomes a policy on the data itself rather than a separate network. Calls, emails, and messages are signed frames, and the economics of spam largely collapse. The architecture scales linearly, shards naturally along the social graph, and changes the cost of attack by replacing anonymous network traffic with signed, attributable assertions.

Neither pillar is novel in isolation; semantic structuring and local-first software each have decades of history. The contribution of this paper is the synthesis. What is new is the claim that both belong in the same layer, as an open commons. The linguistic resources, cryptographic infrastructure, and engineering tools to build it exist now in a way they did not a decade ago. This paper develops the architecture, honestly examines what remains unproven, and makes the case that the time to build it is now.

1. The Semantic Void

More than 80 years ago, Vannevar Bush described the central problem of information management:

> The summation of human experience is being expanded at a prodigious rate, and the means we use for threading through the consequent maze to the momentarily important item is the same as was used in the days of square-rigged ships. (Bush, 1945)

Today, the maze is incomparably larger, and the threading, while faster, is no less indirect. The reason is structural: it sits below every tool we build on top of it, and it has been there since the foundational layers of modern computing were laid down.

Consider what happens when a user saves a photograph. The filesystem records bytes at a path. The operating system tracks the file's size, its modification time, its location on disk. The image format encodes pixels and, sometimes, a small amount of EXIF metadata: camera model, GPS coordinates, exposure settings. Yet no layer of the stack knows what the photograph *is*. Who is in it. What occasion it documents. How it relates to other photographs, to the people depicted, to the day it was taken. That knowledge exists only in the user's head, or in the proprietary database of whichever application the user happens to use. The stack has stored the photograph without storing any of what makes the photograph matter. This is not specific to image files. It is a structural property of every layer beneath the application:

- **The filesystem** sees bytes at paths. Whether those bytes are a novel, a spreadsheet, or a genome sequence is invisible to the layer that stores them.
- **The operating system** sees processes, file descriptors, and memory pages. It cannot distinguish a medical record from a restaurant menu.
- **The network layer** sees packets with source and destination addresses. HTTP adds content-type headers, which identify *format* (`text/html`, `application/json`), never *meaning*.
- **The database** sees rows and columns, or documents and collections. Its schema is local to the application that defined it. Two databases storing information about the same person share no vocabulary for describing what they hold.
- **The web** sees pages at URLs. Search engines exist precisely because the web cannot answer "what is this about?" Third parties crawl billions of pages, guess at meaning from word frequency and link structure, and sell access to their proprietary guesses.

Each layer was designed for generality, and each achieves it the same way: treat data as opaque and leave interpretation to the layer above. This is a defensible engineering choice when generality is the goal, but the cumulative consequence is that meaning has no home in the architecture. Every application that needs to do anything with meaning must build its own semantic layer from scratch: its own schema, its own vocabulary, its own query logic, its own integration adapters. The cost is immense but invisible, because it is paid everywhere and attributed nowhere.

The cost takes familiar forms. Every API integration is a bespoke translation between two systems that cannot describe their own contents to each other. Every search engine is a probabilistic compensation for the fact that no layer of infrastructure records what data means. Every data migration is an exercise in reconstructing meaning that was present in the creator's mind but never captured by the infrastructure. The most recent and most expensive compensation is the large language model. The current wave of AI is, at its core, an effort to recover meaning from data that never stored it. LLMs are trained on vast corpora of human text precisely because meaning lives in the text, not in the infrastructure that holds it. Many of the tasks they perform (classification, extraction, translation, summarization, relationship discovery) are tasks a semantic layer would make trivial, because those tasks reduce to structured lookups when meaning is already encoded in the data's structure. We are building ever-larger statistical models to guess at what could have been recorded directly.

A smaller but revealing instance of the same pattern appears wherever applications exchange key-value pairs, which is nearly everywhere. Configuration files, HTTP headers, database rows, JSON objects, environment variables: the key-value pair is the most fundamental composable pattern in computing. Yet because keys are application-defined strings, they are fractured beyond repair. One system's `author` is another's `creator`, another's `created_by`, another's `dc:creator`, another's `writtenBy`. They all mean the same thing, yet no layer of infrastructure knows this. The fragmentation is not cosmetic, it is what makes integration between systems one of the hardest and most persistent challenges in the industry.

Anyone who has worked with databases has encountered one of the great unsolved problems of the digital age: how to store a physical address. Every schema handles it differently: `address1`, `address2`, `city`, `state`, `zip` is the common American attempt, but it falls apart immediately for addresses in Japan (which are structured by prefecture, district, and block), Germany (where the street number follows the street name), or rural areas in

many countries (where street numbers may not exist at all). Decades of software engineering have not produced a universal address schema, because the problem is not the schema. The problem is that each component of an address (street number, postal code, prefecture, floor) is a *meaning*, and no layer of the infrastructure can represent meanings.

When no layer below the application stores meaning, meaning must live somewhere, and the only place left is application code. An application's schema and interpretation logic become the sole route from bytes to whatever users actually care about. Users do not care about bytes. They care about photographs, messages, relationships, purchases, conversations, every one of which is a structured interpretation. Whoever holds the code that produces the interpretation holds the thing users actually value. Data without an application that understands its schema is bytes without value.

Exporting data is therefore possible but insufficient. Standard export mechanisms (GDPR requests, JSON dumps, CSV downloads) deliver bytes accompanied by a schema that is meaningful only within the originating application's conventions. Another application can interpret those bytes only by reconstructing that schema, a translation that is expensive at scale, lossy at the edges, and bespoke per platform. Users can leave nominally but not practically. The gap between nominal and practical exit is what we experience as platform ownership. It is not a policy that could be legislated away by mandating more export formats, it is the shape of the substrate. Every data-portability regulation confronts the same underlying constraint: bytes move, meaning does not.

A semantic base layer would alter this equation at its foundation. If data is structured by meaning at the moment of creation, any application that understands the shared vocabulary can interpret it, and exit cost approaches zero because switching applications no longer means losing your work. The bundle of data, schema, interface, social graph, and moderation that platforms sell as one thing unbundles, because each component becomes expressible in the same substrate. What we call the platform's moat, the structural advantage that makes leaving impractical, does not survive that unbundling; it dissolves not through regulation or competition but because there is nothing bundle-specific left to own.

That is the first of two structural gaps that together produce the platform era. The other operates at a different layer of the stack, and its emergence is more recent.

2. The Age of SaaS

Consumer computing has passed through two completed eras and is deep into a third. The trajectory has not been linear. The current moment is, in a specific sense, a return to something the industry already left behind, with the critical difference that the centralization is now a commercial choice rather than a hardware necessity.

In the mainframe era of the 1950s through the 1970s, compute was centralized because it had to be. The hardware was expensive, rare, and physically immobile, and users reached it through terminals with just enough intelligence to negotiate a session. All real work happened in the machine room. This was not consumer computing in any meaningful sense; mainframes were institutional. Still, the arrangement established a pattern: compute in one place, access from another, the asymmetry between them non-negotiable.

The personal computing era of the early 1980s through the mid-1990s was the first time ordinary people could run real software on their own machines. The IBM PC, the Macintosh, and their peers put meaningful compute on individual desks. Software came on floppies and later on CDs, installed locally, and ran locally. The user's data lived on the user's disk. The software itself was almost always proprietary, closed-source, and licensed rather than owned, but the locus of computation and state was local. The user ran the software, held the data, and controlled when the software ran, and when it stopped. The vendor controlled the code; the user controlled the operation.

The networked era began in the mid-1990s and continues in steadily deepening form. The web started as a document-retrieval system and evolved into an application-delivery system through incremental accretion: HTML learned forms; JavaScript learned to manipulate the page; AJAX learned to talk back to servers without reloading. The browser slowly became a runtime, but the runtime's job was to render what a server decided, and each accretion moved the locus of computation further from the user's machine.

What began as hypertext documents has become software as a service (SaaS). Applications are no longer something you run, they are something you access. They live in data centers you do not operate, on

infrastructure you do not own, under subscriptions you do not escape without losing access to your own work. Nearly every productive tool a modern user touches (email, documents, design work, communication, project management, calendar, photos, notes, code hosting, source-control hosting, payment processing) is a service running on someone else's computers. Even the tools that retain a local presence, like Slack or Discord or Photoshop or 1Password, are clients for remote services, useless if the service goes away.

The web protocol carries part of the responsibility for where this has led. HTTP was designed for document retrieval: a client asks a server for a thing, the server sends it. Everything interactive we have built on that foundation has preserved the direction of authority. The server holds state, runs logic, and decides what to send; the client renders. JavaScript in the browser has closed some of this gap, but the direction is unchanged: the client's code is code the server chose to ship, running against data the server chose to expose. When the server goes away, the client becomes decoration.

The most powerful consumer computers ever built are being used as near-dumb terminals. The phone in a user's pocket in 2026 has more processing, more memory, more storage, and more bandwidth than the mainframes of the 1970s. A modern laptop has more capacity than most users' everyday workloads come close to using. Yet very few consumer-facing applications actually run on these machines; the device renders what a server farm computes. The sophistication of the hardware is spent on rendering engines rather than on the work the user came to do. This is not a technical outcome but a business outcome, imposed on users whose hardware could do vastly more if anyone would ask it to.

Running the computation is how you own the decisions, and the set of decisions a server makes on a user's behalf is larger than users typically consider. Recommendation order, feed curation, content moderation, search ranking, fraud detection, A/B test assignment, price discrimination based on account signals, the shape of what is shown and the timing of when it is shown, the filtering of what is hidden. All of this happens on servers, inside code users cannot inspect, on data users cannot access, subject to policies users did not agree to and cannot amend. Even if a user's data were semantic and portable, the decisions made about that data would still belong to whoever was running the application logic. Platforms own not just the meaning of data but the agency over it.

Data opacity and the SaaS era are complementary structural causes of platform ownership, and fixing only one is insufficient. Semantic data trapped on server-side infrastructure is no more portable than opaque data on server-side infrastructure. Local compute operating on opaque data is no more powerful than remote compute operating on opaque data. Together the two produce the condition users experience in the current era: your data means nothing without the application, and the application is not something you run.

What is missing is not a better search engine or a smarter parser, and not a different platform hosting the same structural pattern. What is missing is a layer where meaning is the fundamental unit of storage, identity, and retrieval, and where that layer lives on hardware the user actually controls. Both gaps must be closed in the same layer, or neither closure is effective.

3. How We Got Here

Neither gap is the result of inattention. The historical conditions that produced them have been understood for decades, and serious attempts to close them have been made on both sides. Each deserves a closer look, because what they share architecturally explains why a new substrate, rather than another addition to the existing one, is the only way forward.

When the foundational layers were laid down in the 1970s, nodes were disconnected and bytes were precious. The byte-stream abstraction (everything is a file, a file is a sequence of bytes) was a practical triumph given the constraints. TCP/IP, HTTP, SQL: each subsequent layer solved the problem in front of it with the resources available. A semantic data model was not rejected, it was beyond the horizon. The centralizing trajectory of the commercial web was even further out.

The semantic gap persisted in part because the linguistic foundations for closing it took decades to mature. A semantic key cannot be a string; it must refer to a stable, language-independent concept with a hierarchy and participation in structured relationships (what Fillmore called "scenes"). Building those objects requires empirical research into how meaning is structured across human languages. The resources that make it tractable

— WordNet (Miller et al., 1993), Collaborative Interlingual Index (Bond et al., 2016), FrameNet (Baker, Fillmore, & Lowe, 1998), VerbNet (Kipper-Schuler, 2005), and ISO 24617-4 (2014) — are products of computational linguistics that have only recently reached the maturity needed to serve as a practical foundation. Once they did, the commercial landscape of the 1990s and 2000s ran in the wrong direction: every major platform held its data models close because controlling the model meant controlling the ecosystem. Interoperability was a competitive threat to the kind of cross-organizational collaboration a shared semantic foundation requires.

The locality gap has a different shape. The hardware became capable enough for serious local computation by the mid-1990s and has only grown more so. What drove computation away from users was not a technical limit but a convergence of commercial incentives: hosting got cheap, network effects rewarded centralization, and the subscription business model worked perfectly for services and not for shipped software. For any product taking shape in the 2000s and 2010s, a hosted service was easier to monetize, easier to update, easier to monitor, and easier to prevent users from leaving. A generation of developers grew up with SaaS as the default mental model rather than as the historical anomaly it is. Local-first and peer-to-peer architectures remained technically viable throughout this period but lacked the commercial pull, carried as a counter-current by specific communities (Kleppmann's academic work, Freenet, Secure Scuttlebutt, IPFS, and others) without displacing SaaS as the industry default.

That ambient state has been the backdrop for a second history: deliberate attempts to retrofit semantic structure onto opaque substrates and decentralization onto server-centric ones. None has become foundational. These efforts, taken together, point at the same diagnosis.

The semantic retrofits

The Semantic Web is the most ambitious attempt on the semantic side. Berners-Lee's vision described a web in which "information is given well-defined meaning, better enabling computers and people to work in cooperation" (Berners-Lee et al., 2001). The technical realization (RDF triples, OWL ontologies, SPARQL queries) is rigorous and powerful. Twenty-five years later, RDF is widely used in specialized domains (biomedical ontologies, library science, government data) but has not become a general-purpose semantic layer. The web remains overwhelmingly opaque bytes at URLs.

RDF's genuine strengths are substantial: a universal graph model, formal inference via RDFS and OWL entailment, a powerful query language in SPARQL. In specialized domains where those capabilities matter, RDF has proven its value. Three structural problems kept it from becoming general-purpose.

First, RDF annotates existing resources. It is layered *on top of* the web, *not built into* it. A web page can exist without any RDF and most do. The semantic annotation is optional, which means it is absent in the vast majority of cases. The cost of creating semantic metadata falls on the producer while the benefit accrues to the consumer: a classic misaligned-incentive problem.

Second, RDF requires the author to commit to an ontology (Gruber, 1995). In practice, choosing and using an ontology correctly is hard. It requires expertise that most content creators do not have and are not motivated to acquire. The Semantic Web effectively asks every web author to be a knowledge engineer.

Third, the annotation is disconnected from the content. The RDF description of a web page is a separate artifact from the page itself. It can become stale, incorrect, or inconsistent without any mechanism to detect the divergence.

Concrete systems built on the full RDF/OWL/SPARQL stack, such as the Open Semantic Framework developed by Structured Dynamics from 2009 onward, confirmed these limitations in practice: technically rigorous, adopted in narrow domains, but unable to achieve the general-purpose uptake the vision required. The project went quiet after 2016.

Schema.org addressed some of these problems by providing a single vocabulary backed by major search engines. Its adoption is broader than RDF/OWL, precisely because it is simpler and because search engines provide a direct incentive (better rankings) for using it. Schema.org remains a metadata annotation regardless: typically JSON-LD embedded in a page's HTML head. It describes pages *about* things, not the things themselves.

Dublin Core, **EXIF**, **ID3 tags**, **OpenGraph**, and dozens of other metadata standards each solve a narrow problem. They do not compose. A photograph with EXIF data and a document with Dublin Core metadata cannot be queried together because they share no vocabulary, no addressing scheme, and no common notion of what "subject" or "creator" means.

The structural lesson from these efforts is crisp: **you cannot make a semantically inert layer semantic by annotating it.** The annotation is always optional, always disconnected from the content, always maintained by a different process, and always expressed in a vocabulary local to one standard or domain. The layer itself remains opaque.

The locality retrofits

A parallel set of attempts has tried to retrofit decentralization and user control onto an infrastructure whose business model depends on their absence. Each of these efforts was built by thoughtful practitioners and has adoption within specific communities. None has become the default.

XMPP (originally Jabber, 1999 onward) was an earlier federation attempt focused on messaging and presence. It saw significant adoption in the 2000s, including as the transport for Google Talk and early Facebook Chat. That adoption proved instructive: once the largest deployments sat in proprietary hands, those operators withdrew, leaving the ecosystem without the network effects that had made federation useful. A cautionary tale about relying on large commercial adopters to carry a federated protocol.

The Fediverse (Mastodon, Pleroma, and other ActivityPub-based systems) provides a federated alternative to centralized social media. Users choose an instance, and their accounts interact across instances via ActivityPub. The model genuinely decentralizes operation: there is no single company whose servers must run for the network to function. Its adoption grew substantially after high-profile disruptions at centralized platforms. ActivityPub is a federation protocol, however, not a local-first substrate. User data lives on the chosen instance's servers. Moving between instances is an operation that often loses history or followers. The instance operator remains a gatekeeper for the user's experience; this approach has distributed the gatekeepers, not removed them.

Solid (Berners-Lee and collaborators, 2016 onward) proposed personal data "pods": user-controlled storage that applications request access to. The direction is sound, but Solid layers atop HTTP and OAuth, and applications still carry proprietary schemas for what the pods hold. Without a semantic substrate underneath, the pod is another storage location rather than a reorientation of where meaning lives.

Secure Scuttlebutt (Tarr et al., 2019) demonstrated a fully peer-to-peer social protocol with cryptographic identity and append-only signed logs. Technically it is closer to what a local-first substrate should do. Adoption required users to manage their own identities and accept a user experience that had not been optimized for mass adoption, however. The community that embraced it has been small and committed, and SSB has not displaced mainstream social software.

Freenet (Clarke et al., 2001) was one of the earliest sustained attempts at a peer-to-peer substrate for publishing and retrieval. It introduced content-addressed storage routed through a distributed overlay, on the premise that data could be held by peers without any central server knowing where it lived. Its design directly influenced much of what followed, and its continued development across more than two decades, including a substantial recent rewrite, demonstrates both that the technical approach is viable and that it still rewards fresh thinking. Freenet has not reached the mainstream because its user experience, content model, and threat-model trade-offs shaped it for a specific community: anonymity-focused publishing, rather than for general-purpose use.

BitTorrent (Cohen, 2003) demonstrated at massive scale that peer-to-peer distribution works when the architecture fits the problem. In its trackerless form, using a DHT, it operates without any central coordinator at all, and it has carried significant fractions of global internet traffic for over two decades. It solves bulk file transfer extremely well, but does not attempt to be a substrate: files remain opaque, with no common data model, identity, or semantic structure.

Git (2005) is the most widely used distributed system in the world and an instructive case: technical decentralization can survive while cultural centralization takes hold anyway. Each clone is a full repository;

content-addressing via SHA-1 (migrating to SHA-256) makes every object identifiable by its content; any repository can sync with any other over any transport; no single server is architecturally required. Yet the surrounding workflow (pull requests, issue tracking, CI, code search, discovery) became the value proposition of GitHub and a handful of similar platforms (GitLab, BitBucket, and others), and the developer community centralized around them despite Git itself being fully distributed. The lesson cuts against complacency about technical decentralization: it is necessary but not sufficient. If the workflow and social layers around the artifact become the real center of gravity, those layers become the point of centralization, and the underlying tool's distributedness does not save users from a new gatekeeper.

IPFS (Benet, 2014) builds on Freenet's lineage, on BitTorrent's chunked-distribution model, and on the broader distributed-hash-table research tradition (Chord, Kademlia, Pastry, Tapestry and their descendants) that made decentralized lookup practical at scale. It provides content-addressed storage and a peer-to-peer distribution network, solving the real problem of moving bytes between peers without a coordinating server. IPFS on its own, however, is a storage layer. The data moving through it is still opaque. Without a semantic substrate, a file retrieved from IPFS is the same bytes one would retrieve from any CDN, with the same interpretation problem.

The AT Protocol (underlying Bluesky) and **Matrix** take different approaches to federation with real trade-offs around identity portability and decentralization. Neither carries data with meaning in the sense this paper means; both remain federated services rather than a reoriented substrate.

Two structural lessons emerge, mirroring but distinct from the semantic one.

You cannot make a server-centric layer local-first by federating it. Federation (Fediverse, Solid, AT Protocol, Matrix) distributes the servers but preserves the client-server boundary. The data still lives on servers; interpretation still requires application code that lives on servers; agency over what happens to the data still belongs to whoever operates the server. The problem was never the number of servers; it was the architectural privilege of being a server.

Peer-to-peer transport solves a real problem, genuinely well. Freenet, BitTorrent, SSB, and IPFS each made real contributions: content addressing, distributed hash tables, incentive-compatible chunk exchange, append-only signed logs. The problem of moving bytes between peers without a central coordinator is solved by them. Their limitation is not in execution but in routing assumption: each treats all data as homogeneous and all peers as interchangeable. Data is stored on nodes whose IDs are mathematically "close" to the content's hash and found by navigating toward that hash through successive hops. A peer is responsible for a piece of data not because it cares about the content but because of a mathematical property of its ID. This works technically, and at impressive scale (BitTorrent's DHT has operated with tens of millions of nodes). But the costs grow with ambition: lookup latency scales as O(log N) hops, each a network round-trip; nodes store data they have no interest in because the hash says they should; churn requires constant routing-table maintenance; and the model assumes all nodes are equivalent when real devices range from phones to servers. More fundamentally, hash-distance routing treats all data the same way regardless of what it is or who cares about it, and in doing so it consumes the flexibility a substrate would need to route different data to different people based on their relationships. Real data is not uniformly relevant. A chess move matters to the players. A photograph matters to the people depicted. A medical record matters to the patient and their doctor. A substrate that cannot see these differences remains a transport layer, not a foundation on which applications become interchangeable.

The common lesson

The three patterns of retrofit converge on a single diagnosis: additions cannot compensate for a substrate whose shape is wrong for the job. Annotation does not make opaque layers semantic. Federation does not remove the client-server boundary. Peer-to-peer transport routes all data as though it is the same and all peers as though they are interchangeable. Each is an addition to an architecture that was not built for what the addition requires, and each falls short in the specific way the underlying architecture leaves no room for it to succeed. Closing either gap requires a different substrate, not another addition to this one.

The solution must be a *layer* where creating data is simultaneously creating semantic structure and locating that data in a user-controlled peer. The two properties are not separate operations, and neither can be supplied by annotation, federation, or transport alone.

What's different now

Four things have converged to make such a substrate possible now in a way it was not before. First, the computational linguistics infrastructure matured: WordNet, CILI, FrameNet, VerbNet, and ISO 24617-4 collectively provide the grounded vocabulary the semantic pillar needs. Second, the technical infrastructure for distributed state matured as well: CRDTs (Shapiro et al., 2011) have become practical, signing cryptography is cheap enough to apply at the per-message level, content-addressing is ubiquitous (every Git commit is a use of it), and local-first synchronization has moved from research topic to shipping practice. Third, global interconnection made shared vocabularies both necessary and viable; the network that makes the semantic problem acute is the same network that makes a collaborative solution practical, and the same network over which a peer substrate would operate. Fourth, the open-source movement created the collaborative environment such a commons requires. The linguistic databases carry permissive licenses. The cryptographic foundations are open. The storage and networking building blocks are open. A base layer along both pillars is inherently a commons: it only works if shared, and it can only be shared if open. That commons is now possible in a way it was not during the era of proprietary platform wars and server-centric default architectures.

4. What a Base Layer Requires

If neither a semantic layer[1] nor a local-first substrate can be achieved by annotating, federating, or moving bytes peer-to-peer on top of what we already have, what must a new foundation look like? The missing foundation would sit above the raw byte substrate that filesystems and networks already provide, and below the application that currently monopolizes interpretation. The answer to what it requires has two halves. The first concerns meaning: what must data be for any application to interpret it? The second concerns locality: where must the data and its interpretation live?

Grounded predicates, not strings

The key-value pair is everywhere in computing, and the key is always a string. One system stores a date of birth as `dob`, another as `date_of_birth`, another as `birthDate`, another as `geburtsdatum`. They all mean the same thing yet nothing in the infrastructure connects them. The root of this fragmentation is that strings have no inherent connection to what they denote. A semantic layer requires keys that carry *meaning*, not just labels. The key must refer to a concept, not a string, and that concept must be shared across systems, applications, and languages.

The problem of vocabulary sharing across systems was formalized in foundational ontology research. Gruber (1995) argued that systems cannot meaningfully share knowledge without committing to shared vocabularies whose terms have agreed-upon meanings, and laid out design principles (clarity, coherence, minimal ontological commitment, and others) for the ontologies such sharing requires. The Semantic Web pursued this insight through URI-identified predicates. URIs, however, are locations, not meanings. They are globally unique, but they do not carry semantic content intrinsically. Two different URIs can denote the same concept (`schema.org/author` vs. Dublin Core's `dc:creator`), and nothing in the infrastructure connects them.

---

1 On terminology: "semantic layer" is already used in the data analytics industry to mean a translation layer between database schemas and business concepts, as in tools like Looker and dbt. What is meant here is different. A BI semantic layer sits above the data and translates queries; the layer proposed here would sit *with* the data, because the data would already know what it means.

What we need are keys that refer to *meanings*: language-independent, application-independent units of semantic content with stable identities. Decades of computational linguistics research have produced extensive catalogs of empirically validated meanings, organized by hierarchy and cross-linked across languages. These resources do not hand us a finished vocabulary with stable identities, but they give us the raw material from which one can be built.

Structured assertions

A flat key-value pair (`author: Tolkien`) captures a single relationship, a simple assertion, but loses the structure that gives it meaning. Who is asserting this? About what? In what capacity? A key-value pair has no structure to express the *kind* of relationship, the *participants* and their roles, or the *context* in which the assertion holds.

What we need is the frame pattern: a **predicate** that defines a structured assertion, and **role bindings** that fill its slots with values. The theoretical foundation for this pattern comes from Fillmore's frame semantics (1968; 1982), and the empirical grounding comes from computational-linguistics research — FrameNet, VerbNet, and ISO 24617-4 — that has cataloged and standardized the thematic roles human languages use to describe who did what to whom, where, when, how, and why.

Write-time resolution

This is the core inversion. The dominant pattern in computing is to store data first and try to determine its meaning later. Some systems capture partial structure through schemas, but those schemas are local to the application; the meaning does not travel with the data. Search engines crawl, natural language processing systems annotate after the fact, data integration pipelines map between schemas post-hoc. All of these are attempts to recover meaning that was present in the creator's mind but never fully captured in the infrastructure.

A semantic base layer reverses this. Meaning could be resolved *at the moment of creation*, when it is trivially easy, because the creator knows what they mean. The disambiguation that search engines and NLP pipelines struggle to perform after the fact is effortless at write time. When a user creates a relationship between a person and a book, they know whether they mean "authored," "edited," "reviewed," or "purchased." If the layer captures that distinction as a grounded semantic predicate at creation time, no subsequent system ever needs to guess.

The predicate, once chosen, tells the system what roles to expect. The system can prompt for them, offer completions, validate inputs. The act of creating data *becomes* the act of resolving meaning, because selecting a predicate and filling its roles is inherently a semantic operation.

This is not natural language understanding. Such a layer need not parse free text and try to extract meaning. It would structure the input environment so that meaning is captured as a natural consequence of creation. The user selects a predicate, fills roles, and the result is a grounded semantic structure. The hardest problem in NLP (disambiguation) is trivially solved at write time by the person who knows what they mean.

Cross-lingual stability

A semantic layer that works only in English is an English-language metadata standard, not a semantic layer. The concept that English speakers call "dog," Spanish speakers call "perro," and Japanese speakers call "犬" is the same concept. A semantic layer must represent meanings independently of the words that express them. That meaning has universal structure across unrelated languages is not merely a philosophical premise: cross-linguistic studies of polysemy have found that the network of which concepts tend to share words is remarkably consistent across unrelated language families, suggesting the organization of meaning is, to a substantial degree, universal (Youn et al., 2016).

This requires a clean separation between *meanings* and *words*. Meanings (which I will call **sememes**, following usage in structural semantics) are language-neutral units with stable identities. Words, in their role as language-specific expressions that point to meanings, are called **lexemes**. Each lexeme belongs to a particular

language and carries the morphological apparatus of that language (inflection, conjugation, case, gender, tense). Multiple lexemes across languages can point at the same sememe: the English “authored,” the Spanish “escrito,” and the German “verfasst” are three lexemes, one sememe. So are “author,” “authoring,” and “authorship” within English, and “verfasst,” “verfassen,” “Verfasser,” and “Verfasserschaft” within German — different word forms, all pointing at the same meaning. The predicate `AUTHORED` exists independently of any of them.

The four requirements above describe data. Four more describe where the data lives and who runs the code that interprets it.

Cryptographic identity

Meanings have identity: the concept `DOG` is the same concept across languages and systems. Participants need identity too, but of a different kind. Users, organizations, devices, services — anything that signs assertions or holds data must be identifiable without depending on a central authority to assign or revoke that identification.

In practice, this means a participant’s identity would be a cryptographic keypair rather than a row in some registry. The paradigm is not new. Pretty Good Privacy (PGP; Zimmermann, 1995) established the foundational design thirty years ago, in which identity is a keypair, trust is a web of signed endorsements rather than a certificate authority, and no central registry is required. The same principles have been developed in the decades since under the banner of **self-sovereign identity** (Allen, 2016): lifetime-portable digital identity that does not depend on any centralized authority. The Rebooting the Web of Trust community (2015-present) has sustained a decade of work in this direction, including the Decentralized Identifiers and Verifiable Credentials standards developed at the W3C. Everyday instances of the same core insight are already in widespread use, from SSH key authentication to the Signal messaging protocol. What all of this demonstrates is that keypair identity is practical and well-understood; what has been missing is a substrate in which it is native, rather than one in which it has been added after the fact.

What PGP demonstrated for email, a base layer could generalize to all assertions. A user should not need to register with a service or keep an account active on a server to exist as a participant. The keypair is something the user holds, generates locally, and presents when they sign an assertion or establish a relationship. No authority grants it; no authority can revoke it (though peers can choose to stop trusting it, which is a social decision, not an architectural one).

This is the minimum condition for agency without a gatekeeper, and the foundation on which everything else in the locality pillar rests. Signatures depend on the signer having a key, relationships depend on the parties being identifiable, and content attribution depends on the assertion being linked to a verifiable identity. All of these build on identity being a thing users hold rather than a thing services grant.

Content-addressed data

In most existing systems, data is named by where it lives: a filesystem path, a URL, a database row ID, a cloud storage key. Move the data and the name breaks. Copy the data and the copies have no way to recognize each other as the same thing. The name is an address, not a fingerprint.

Content-addressing (Merkle, 1979; Benet, 2014) reverses this. Data would be named by a hash of its content, a cryptographic fingerprint derived from what the data *is* rather than where it happens to be stored. The same data retrieved from any source produces the same fingerprint, so copies are recognizably identical and integrity is verifiable locally. This is what decouples data from storage location, which is in turn what makes data portable between peers without losing its meaning, provenance, or relationship to other data. Storage would become commodity; hosting would become a user-revocable choice rather than a platform commitment.

“Social networking”

Identity is a keypair and data is content-addressed, yet neither on its own determines the shape of the network. Where does data live? How does it reach the people who care about it? How do users find data they do not yet know exists? Who can see what? In a centralized architecture, a server answers all of these: it stores the data,

routes requests, curates discovery, and gates access. A substrate without servers needs a different organizing principle.

As described earlier, peer-to-peer systems have mostly answered these questions through hash-distance routing, which works technically but treats all data as homogeneous and all peers as interchangeable. A different organizing principle is needed, one that reflects a basic fact: data is not uniformly relevant. Every person cares about specific data — their own work, data from people they know, data about topics or places or communities of interest to them. Relevance follows relationships: between individuals, between organizations, between communities, between devices and the people who use them.

If relevance follows relationships, then relationships should organize the network. Data would live with the peers who care about it, travel along the connections that make it relevant, and surface to new users through shared relationships rather than algorithmic curation. Access would follow from whom the holder chose to include. Sociologists have studied such structures under the name *social networks* for more than half a century (Barnes, 1954; the social-network-analysis tradition that followed), as the shape through which information, influence, and resources actually flow among humans. The commercial platforms that adopted the phrase "social networking" in the late 1990s and 2000s built closed enclosures around a small slice of this phenomenon and sold access to it. The substrate proposed here is aimed at the underlying structure rather than any particular enclosure of it: a network whose topology is the social graph itself, not a product that owns and resells it.

The claim that social structure can serve as an organizing principle is not just philosophical. Human social networks empirically exhibit *small-world* properties (Milgram, 1967; Watts & Strogatz, 1998): despite their sparseness, most pairs of people are connected through a small number of intermediate relationships. Data stored and discovered along the social graph can therefore reach most users through a handful of hops on average, even at scale. This does not mean the social graph is a perfect topology. Communities can be insular, trust circles can overlap unevenly, and some users may sit at the edges of the network with few connections. The claim is not that social structure eliminates all routing and discovery problems, but that it is a better organizing principle than either centralized servers or socially-blind distance metrics, and that the gaps it leaves are addressable through mechanisms (well-connected community peers, public directories, opt-in discovery services) that complement rather than replace the social structure underneath.

Users of today's centralized services store, discover, and access data through relationships inherited rather than chosen: the employer's cloud, the default search engine, the platform where the relevant people already happen to be. Replacing inherited structure with deliberate choice, expressed in the substrate itself, is the shift.

The social graph, used as the organizing principle, answers the questions this section opened with at once: data lives with the peers who have relationships with it (storage), travels along those relationships when needed (routing), surfaces to new users through shared connections (discovery), and remains visible only to those the holder chose to share with (access).

Anonymity is a legitimate need: users sometimes want to obscure their identity, location, or routing paths. Routing along the social graph does not preclude this — it can serve the purpose directly. What today requires separate, purpose-built networks (VPNs, Tor) could instead be routing choices within the same substrate: direct peer connection, relay through a trusted intermediary, or multi-hop paths with layered encryption so no intermediate peer sees both origin and destination — all governed by the same trust relationships that organize everything else.

Local execution

The previous three requirements address the data: how it is signed, how it is named, and how it is organized across the network. The fourth addresses the code that interprets it.

In the SaaS model, the runtime lives on the server. The user's device receives the results of computation it did not perform, rendered into a form it cannot inspect or modify. The application is something the user accesses, not something the user runs. As we saw earlier, this is where decision-ownership lives: recommendation, curation, moderation, pricing, filtering all happen server-side, inside code users cannot see.

Local execution reverses this. The runtime that interprets data and acts on it would run on the user's machine. The user's device would be a full participant rather than a renderer, and the computation that turns

data into experience would happen there. This does not mean every computation must be local (some workloads require specialized infrastructure), but it means the *default* is local, and remote computation is an explicit delegation rather than the only option. The user's relationship to their own data stops being mediated by code they do not control.

These requirements do not name a structure, but rather constrain one. Whatever fits has to be built around meaning rather than strings, carry role-keyed values rather than flat attributes, be complete at the moment of writing, survive translation between languages, be identified by content rather than location, carry identity through keypairs rather than server accounts, route along deliberately-chosen relationships rather than through central brokers, and execute locally rather than remotely. None of these requirements is individually novel. What would be new is asking a single structure to satisfy all of them at once, and to do so as the foundation of a layer rather than an annotation or federation laid over existing layers.

5. The Frame as Primitive

The preceding sections diagnosed two structural gaps and laid out what closing them would require: eight properties, four semantic and four locality, that a new substrate must satisfy simultaneously. The diagnosis is complete. What follows is a proposal: a specific primitive and the architecture that grows from it. The primitive is not the only shape that could satisfy the requirements, but it is the one this paper develops, and it has the advantage of being grounded in half a century of empirical linguistics rather than invented from scratch.

Fillmore's *semantic frame* (1968; 1982) was designed for a different purpose: analyzing what sentences mean rather than structuring data. Using it as a data primitive is an approach the linguistic literature never had occasion to consider. A semantic frame, in this usage, is:

```
Frame {
    predicate:  a grounded meaning              (what kind of assertion)
    bindings:   compound-key → value pairs      (the semantic content)
}
```

A predicate and its bindings. Nothing else is structurally required. Each binding maps a **compound key** to a value. A compound key is a sequence of grounded meanings that together identify what the binding *is*. A key might be a single role like `(AGENT)`, or a qualified sequence like `(VALUE, ENGLISH)` where additional meanings narrow the role with arbitrary precision. Every element of the frame (what it asserts, what it is about, who is involved, what content it carries) is expressed as a binding on the predicate.

A frame is not a key-value pair with extra structure. It is a coherent assertion whose roles are determined by its predicate. A title requires something being titled and the title itself. An authorship assertion requires a work and an author. A chess move requires a game, a player, a piece, a source square, and a destination square. Remove any of these and the assertion does not make sense, the way "John gave" does not make sense without knowing at least what was given and to whom. The predicate declares what roles are needed for the assertion to be complete, and the roles are not arbitrary labels but grounded meanings that name the function each value serves.

A **title assertion**: `TITLE { (THEME) = the-book, (VALUE, ENGLISH) = "The Hobbit" }`. A separate frame carries the Russian title: `TITLE { (THEME) = the-book, (VALUE, RUSSIAN) = "Хоббит" }`. These are two independent assertions, each separately signable, because a translator should not need the original signer's key to add a translation. The compound key `(VALUE, RUSSIAN)` distinguishes the binding from `(VALUE, ENGLISH)` through the language qualifier.

A **chess move**: `MOVE { (LOCATION) = the-game, (AGENT) = Fischer, (THEME) = king-pawn, (SOURCE) = e2, (GOAL) = e4 }`. Location (which game), Agent (who moved), Theme (what piece), Source (from where), Goal (to where). A single move is a single semantic assertion.

A **definition**: `GLOSS { (THEME) = the-sememe, (VALUE, ENGLISH) = "a domesticated canine" }` and separately `GLOSS { (THEME) = the-sememe, (VALUE, SPANISH) = "un canino domesticado" }`. Same pattern as the title: each language's gloss is its own independently-authored frame.

An **authorship assertion**: `AUTHORED { (THEME) = The Hobbit, (AGENT) = Tolkien }`.

These are all structurally identical: a predicate and role bindings. The predicate determines what roles the frame expects; the roles determine what the values mean.

The predicate is worth pausing on, because it is easy to treat it as a distinct kind of thing. It is not. It is a sememe, a unit of meaning from the shared vocabulary, acting in a particular structural role. In that role, a sememe serves as a template or schema for the frame: it declares what bindings a frame of this kind is expected to carry and how those bindings relate. Calling a sememe a *predicate* names the role it plays, not a category it belongs to. `AUTHORED`, `TITLE`, `MOVE`, `BOOK`, and `DOG` are all sememes, and which of them naturally fits the predicate role is a matter of what each one denotes, not a structural constraint. Sememes that name relations or events (`AUTHORED`, `TITLE`, `MOVE`) naturally fit as predicates. Sememes that name kinds of thing (`BOOK`, `DOG`, `CHESS`) fit instead as templates for items. A specific instance (J.R.R. Tolkien, a particular dog named Rover, a single chess game played on a Tuesday) is an item in the graph but not a sememe: it is an instance of a sememe rather than being a meaning in its own right.

Compound keys and indexing

The title and gloss examples above demonstrate compound keys in action: `(VALUE, ENGLISH)` and `(VALUE, RUSSIAN)` use a language qualifier to distinguish bindings that share the same base role. The same mechanism extends to any domain: `(VIDEO, MKV, UHD)` and `(VIDEO, MKV, HD)` would distinguish video resolutions within the same container format; `(CONFIG, REPLICATION)` and `(CONFIG, RETENTION)` would distinguish policy types. Every element of a compound key is a grounded sememe, not an arbitrary string.

There is one controlled exception. The first element of a compound key must always be a sememe, grounding the binding in the shared vocabulary. Subsequent elements, however, can be **literals**: user-defined names that serve as qualifiers within the scope of the item. A binding keyed `(THEME, x)` uses the sememe `THEME` as its grounded root and the literal `x` to distinguish this particular subject from others on the same item. The literal `x` has no global meaning; it is scoped to the item that carries it, the same way a variable name is scoped to the function that declares it. This is what makes variable binding, cell addressing, and local naming possible within the frame model. A small example shows how this works in practice:

```
EQUALS { (THEME, x) = 5 }
EQUALS { (THEME, y) = ADD { (THEME) = x, (INSTRUMENT) = 3 } }
```

Two frames on the same item. The first defines the variable `x` as 5; the second defines `y` as `x + 3`, referencing `x` by name. `THEME` is the root because `x = 5` is *about* x — x is the subject being defined. `x` and `y` are literals scoped to this item — arbitrary variable names that no vocabulary would catalog. `EQUALS`, `ADD`, `THEME`, and `INSTRUMENT` are sememes from the shared vocabulary. The local names let you build up computations that reference each other; the semantic roots ensure that the structure is always grounded. The consequence is that every item is, in a sense, a spreadsheet: a structured workspace where named values can reference each other, with the semantic root ensuring that the *kind* of value is always grounded even when the *name* is local.

Every sememe in a compound key is an opportunity for indexing (literal keys, being locally scoped, are not indexed globally). A layer that indexes frames by the grounded meanings in their binding keys gets multi-dimensional search as a structural consequence: "show me all videos" is a lookup on frames with `VIDEO` in their keys; "all UHD videos" narrows to frames with both `VIDEO` and `UHD`. No separate tagging system, no search facets, no metadata catalog. The key *is* the index.

Queries are frames

The indexing infrastructure described above has a natural consequence: queries are not a separate mechanism. *A query is an incomplete frame*, a frame with one or more bindings left unfilled, and the unfilled bindings indicate what you are asking for.

`AUTHORED { (AGENT) = Tolkien }` with no `THEME` binding asks "what did Tolkien author?" `DEPICTS { (VALUE) = sunset }` asks "what depicts a sunset?" `MOVE { (LOCATION) = the-game }` asks "what moves were made in this game?" In each case, the system matches the filled bindings against the index and returns frames that complete the pattern.

Any binding in a query can hold an **expression**: a sub-frame that evaluates rather than matching literally. `LISTING { (THEME) = book, (VALUE, PRICE, USD) = LESS_THAN { (VALUE) = 20 } }` uses a `LESS_THAN` sub-frame as a filter on the price binding. The sub-frame is itself a frame with its own predicate and bindings, composed the same way assertions are composed.

Queries can also be **compound**: multiple incomplete frames joined by shared references. "Find books authored by Tolkien that have a listing under $20 USD" requires defining a variable, constraining it with two patterns, and letting the query machinery find what satisfies both:

```
EQUALS   { (THEME, book) = ANY }
AUTHORED { (AGENT) = Tolkien, (ANY) = book }
LISTING  { (ANY) = book, (VALUE, PRICE, USD) = LESS_THAN { (VALUE) = 20 } }
```

The first line defines `book` as a literal variable (using the same literal-key mechanism described above) bound to `ANY`. `ANY` is not special syntax; it is a sememe in the shared vocabulary, a meaning that resolves to "match anything" the same way `LESS_THAN` resolves to a comparison. Any expression could appear in its place to constrain the match differently. The second line constrains the variable: `book` must appear in an `AUTHORED` frame where Tolkien is the `AGENT`. The third constrains it further: the same `book` must appear in a `LISTING` frame with a price under $20. `ANY` as a role means "I don't care which binding — just find frames that reference this item." The query machinery finds assignments to `book` that satisfy both patterns simultaneously.

A frame is an assertion when every binding is filled, and a query when at least one is left open. There is no separate query language, no SQL, no SPARQL, no GraphQL. The frame IS the query, the shared vocabulary IS the schema, and the compound-key index IS the query engine. This is arguably the most powerful consequence of using a single semantic primitive for everything: the ability to ask questions about data is not a feature bolted on after the data model is defined, but a structural property of the data model itself.

Everything is a role binding

There is no fundamental distinction between "data" and "metadata" of a frame. Each is a value filling a role. A title's text, a video's master file, a chess move's destination square, a document's author: each is a role binding. Provenance, signatures, and timestamps are all bindings. The distinction is conventional, not structural.

Collapsing it has consequences. In conventional systems, metadata lives in sidecar structures — EXIF, xattr, audit logs, file headers — each with its own subsystem, query pattern, and lifecycle. When everything is a frame, those subsystems collapse into one. A query for who authored an item and a query for what it contains use the same mechanism. And because every statement about an item is a frame, anyone with something to add (a review, a fact-check, a translation) can contribute as a first-class assertion rather than as an afterthought riding in a separate channel.

Beyond the natural-language inventory

The 22 thematic roles inherited from linguistics (drawn from VerbNet and ISO 24617-4) describe the participants in events: who did what to whom, where, when, how, why. For a frame primitive that has to stand

in for everything a data layer stores, they are necessary but not quite sufficient. Three additional roles emerge as soon as the frame is asked to do work that natural language did not need to do.

The first gap appears as soon as a frame needs to carry raw content. An `IMAGE` frame's JPEG bytes, a `GLOSS` frame's definition text, a `MEASUREMENT` frame's numeric result, a `TITLE` frame's designation: none of these are Agents, Themes, or Goals. They are payloads, the content a predicate asserts rather than a participant in an event. The standard role inventories have no general-purpose role for "this is the content itself." **`VALUE`** generalizes VerbNet's narrow Value role (which covers only scalar endpoints like prices) into the role for whatever a predicate carries as its payload: a designation, a quantity, a measurement, a piece of text, a binary blob.

The second gap is operational. How should an assertion be handled once made? Replicated? Encrypted? Presented? Retained for how long? **`CONFIG`** is the role for policy on the assertion itself: `(CONFIG, REPLICATION)`, `(CONFIG, PRESENTATION)`, `(CONFIG, RETENTION)`, and so on. Any frame can carry `CONFIG` bindings regardless of its predicate.

The third gap is causal ordering. A chess move follows the previous move; a paragraph edit follows the edit before it; a reply follows the message it answers. **`FOLLOWS`** is the role for causal or temporal predecessors, and like `CONFIG` it is cross-cutting: any frame can carry a `FOLLOWS` binding pointing at the content hash of an earlier frame. The result is a hash-linked chain of signed assertions, local and contextual (this game, this conversation, this document) rather than a global consensus mechanism.

Three roles added to the linguistic inheritance: `VALUE` (generalized), `CONFIG` (new), `FOLLOWS` (new). Each names a function the linguistic literature had no need to catalog.

Frames as portable units

A frame carries within itself everything needed to interpret it *structurally*: its predicate names the kind of assertion, its binding keys name what each value means, and the values carry the content. No external schema is required, no application-specific decoder ring, no lookup against a central authority. Understanding what the predicate and roles *mean*, however, requires holding those sememes in your vocabulary. If a frame arrives referencing a predicate or role you have not encountered before, you fetch it from a peer who has it, the same way you would fetch any other item. An application can define new predicates for its own functionality, new kinds of items as targets for its frames, and even new roles if the standard inventory does not cover its needs, and those extensions propagate through the vocabulary as items like everything else.

A frame retrieved from any source is legible on arrival to any runtime that holds the relevant vocabulary. That self-sufficiency is what makes the frame a unit of transit between peers as well as a unit of storage. Signed, it is a verifiable assertion that can travel through any route and arrive intact. Content-addressed by the hash of its predicate and bindings, it is identifiable as the same frame regardless of where it was retrieved from. Compactly encoded, frames are small on the wire and in storage. These properties are not an addition to the frame primitive; they are consequences of what the frame already is.

Predicates carry behavior

So far we have treated a predicate as a structural template, declaring what bindings a frame expects. A predicate can do more. It can also declare how frames of its kind behave: how they might be expressed in text or other input, how they might be evaluated if they carry a computation. These declarations would be data on the predicate itself, not rules maintained by a separate parser or interpreter.

Consider the token `+`. In ordinary treatment, `+` is a symbol that a language's grammar rules know how to parse. In the proposed frame-based layer, the picture is different. `+` is not a meaning. It is a *token*, a written symbol used in some notations. Other notations use different tokens for the same idea: "plus," "más," "加える." All of them point to the same underlying meaning, the sememe `ADD`. That sememe, in its role as a predicate, can declare the properties a parser would need to know about it: it is infix, it has a precedence, it associates left-to-

right. These properties would not be grammar rules the parser has to be told in advance. They would be data the parser reads off the sememe once it resolves it from a token. No separate precedence table. No grammar.

This extends to structural symbols. Parentheses are tokens whose corresponding meanings declare "I open a group" and "I close a group." There is no reserved syntax. Everything (verbs, operators, functions, parentheses, commas) would resolve through the shared vocabulary. Syntax becomes vocabulary.

Any domain can bring its own notation. Chess algebraic notation, regular expressions, mathematical symbols: each is a set of tokens whose corresponding predicates declare how they parse, resolved through the same mechanism as arithmetic operators or English prepositions.

The behavior a predicate declares is best understood as a *contract*, not a piece of code. The contract lives with the predicate in the vocabulary; code that satisfies the contract (an actual parser, an actual evaluator) is something else entirely. Where such code comes from and how it gets attached to a predicate is a question the primitive itself does not answer.

6. What Frames Cohere Around

Frames are the primitive, yet a single frame is rarely the whole story. A book is described by `TITLE` frames (in various languages), `AUTHORED` frames (possibly more than one author), `TEXT` frames (the chapters), `COVER_ART` frames (different editions have different covers), `PUBLICATION` frames (different editions, different publishers, different years), and more. Each frame is a separate assertion with its own predicate and bindings, and they are all *about the same thing*. They only make sense together.

If frames can be about the same thing, they need a shared anchor to point to. That anchor, and the frames cohering around it, is what we will call an **item**.

An item is not a new primitive in the way a frame is. It is what falls out when frames need to be *about* something: a stable anchor that frames can reference to indicate "I am about *this thing*." The book is an item. Tolkien is an item. Chess as a concept is an item, and so is each individual chess game. A user is an item, described by `NAME`, `AVATAR`, and `PUBLIC_KEY` frames, referenced as the `AGENT` in every assertion they sign. The local runtime that manages a user's items and peer connections is itself an item. There is no separate notion of "user account"; there are only items with keypairs. Each exists as an anchor around which frames cohere, building up a coherent, multi-faceted description. Any binding on a frame can reference an item, and all such references are equal: `THEME` in an authorship assertion, `LOCATION` in a chess move, `AGENT` in a player registration, `RECIPIENT` in a recommendation. The frame relates to each referenced item through whatever role the binding carries.

Earlier we required that data be named by what it is rather than where it lives, and items satisfy this requirement. Each item has a stable, location-independent identifier, its **item ID (IID)**, that functions the same way regardless of where the item is stored. The IID is not assigned by a central registry; in most cases it is generated as a unique random value, though a small number of foundational vocabulary items use deterministic IIDs derived from canonical keys so they can be referenced in code. In either case, the same IID is recognized on any device, by any peer, without coordination. The same item can exist in many places and still be identifiable as the same item.

New frames bind to an item over time. Some are assertions the item's maintainer chooses to endorse (a new chapter in a book, a new move in a chess game); others are assertions by third parties that bind to the item without being endorsed (a reaction, a review, a fact-check). Both kinds reference the same IID; endorsement is a separate decision.

The set of frames an item endorses at any point in time is recorded in its **manifest**: a signed list of frame endorsements. The manifest is itself content-addressed: its hash is the item's **version ID (VID)**. When the maintainer endorses a new frame, a new manifest is produced, with a new VID, while the IID remains stable. Version history is a directed graph of manifests, each pointing at one or more predecessors, structurally similar to Git's commit graph. The similarity goes further: an item can be forked (a new maintainer starts endorsing a divergent set of frames under the same or a new IID) and merged (two divergent manifest chains are reconciled into one). Peers holding the same item may hold different versions, and synchronization is a matter of

exchanging the manifests and frames each is missing. Conflict resolution when two peers have diverged is an area of active design, informed by CRDT research and by Git's own practical experience managing exactly this problem.

The archetype

What makes a particular item the kind of thing it is? A book is recognizably a book, not because some authority declares it so, but because the frames that describe it are the frames a book is *expected* to have: a title, one or more authors, chapters of text, publication history. That collection of expectations is what "book" means as a category of item.

As described earlier, a sememe playing the predicate role serves as a template for a frame. `BOOK`, `CHESS_GAME`, `PERSON`, `LANGUAGE`, and `DOG` play a parallel role at the item level: they serve as templates for items, declaring what frames an instance is expected to endorse. Call a sememe playing this role an *archetype*. Predicate and archetype are two of the structural roles a sememe can play, both drawn from the same vocabulary.

One important asymmetry between them shapes how items grow. A predicate's declaration is mostly closed: a frame using a particular predicate carries the bindings the predicate calls for, though cross-cutting structural roles like `CONFIG` can appear on any frame regardless of predicate. An archetype's declaration is *open*: it lists the frames an instance is expected to endorse, but anyone can assert additional frames that bind to the item without the archetype having defined them. A book is expected to endorse `TEXT`, `TITLE`, and `AUTHORED` frames. Nothing prevents someone else from signing a `LIKE` frame, a citation, a fact-check, a translation, or a review that binds to the same book. The archetype defines what makes something a book. It does not gatekeep what others may say about it.

A chess game makes the pattern vivid. The game is an item. `CHESS` is its archetype, declaring the frames a chess game is expected to endorse: the players, the moves, the result. The item itself, however, is not a monolithic structure. It is an anchor around which signed frames cohere.

Players would register by signing their own `PLAYER` frames: `PLAYER { (AGENT) = Fischer, (VALUE) = WHITE }` signed by Fischer and `PLAYER { (AGENT) = Spassky, (VALUE) = BLACK }` signed by Spassky. Each player attests their own participation; it is not assigned by a third party but declared by the participant and carries their signature.

Then moves: `MOVE { (LOCATION) = the-game, (AGENT) = Fischer, (THEME) = king-pawn, (SOURCE) = e2, (GOAL) = e4, (FOLLOWS) = previous-move }` signed by Fischer. Each move is independently meaningful, independently signed, independently verifiable, and linked to its predecessor by `FOLLOWS`. The game's move order is not maintained by a separate sequence number or a server's database; it falls out of the `FOLLOWS` chain, the same hash-linked structure described earlier.

Because each move is a self-contained signed frame, the game can be played peer-to-peer: each move travels directly from the player who made it to the other, with no referee server, no hosted backend, no central authority. The game is recorded by being signed; it does not need any third party to become real. And because each move is a frame, it is queryable: "all games where someone opened e4" is a lookup on `MOVE` frames with `(GOAL) = e4` whose `FOLLOWS` binding points at the game's initial state (meaning it was the first move).

The pattern generalizes immediately. A chat room would be an item with signed `MEMBERSHIP` and `MESSAGE` frames. A key log would be `KEY`, `REVOKE`, and `DELEGATE` frames. An auction would be signed `BID` frames. Even real-time shared presence fits: a `PRESENT` frame asserts "I am in this space," and an `AVATAR_STATE` frame whose `CONFIG` policy says "keep only the latest" carries position at 60Hz, replacing the previous one each time rather than accumulating. A chess move is retained permanently; an avatar position is discarded when a newer one arrives; a video feed binds to a content stream whose blocks are consumed and released. The difference is the `CONFIG` policy on the frame, not a separate mechanism. All follow the same pattern: an item exists, people make signed assertions about it, and those assertions collectively define what it is.

The photograph, revisited

This paper opened with a photograph: a user saves it, and no layer of the stack knows what it is, who is in it, what occasion it documents, how it relates to other photographs or to the people depicted. In the picture being described here, a photograph is an item. Its archetype is `PHOTOGRAPH`. The information that today lives only in the user's head, or in a proprietary application's database, would be captured as frames at the moment of creation.

The photographer takes a picture of Alice and Bob at Alice's graduation. The item is created, and with it, the frames:

```
IMAGE {
     (THEME) = the-photo,
     (VALUE, JPEG) = <image bytes>,
     (VALUE, JPEG, THUMBNAIL) = <thumbnail bytes>
}
DEPICTS {
     (THEME) = the-photo,
     (VALUE) = Alice,
     (LOCATION) = [120px, 340px, 280px, 510px]
}
DEPICTS {
     (THEME) = the-photo,
     (VALUE) = Bob, (LOCATION) = [400px, 320px, 560px, 500px]
}
OCCASION { (THEME) = the-photo, (VALUE) = graduation }
PLACE    { (THEME) = the-photo, (LOCATION) = [37.4275°N, 122.1697°W] }
CAPTURED { (THEME) = the-photo, (INSTRUMENT) = iPhone, (TIME) = 2024-06-15 }
```

Each frame is a separate assertion, signed by the photographer and indexed by the meanings in its binding keys. The photograph does not merely *have* this information attached to it as metadata. The frames ARE the photograph's semantic content, each independently signed and each independently verifiable.

The photograph lives on the photographer's device, managed by her **Librarian**: the local runtime that stores items, endorses frames, maintains relationships with other peers, and handles replication. Every participant in the network runs a Librarian; it is the software that makes the substrate real on a given device. Alice and Bob, who were depicted, have a relationship with the photographer; her Librarian replicates the photograph to theirs because she has chosen to include them. Alice's mother, who follows Alice, sees the photograph arrive on her own device because Alice chose to include her. No upload to a central server occurred. No platform decided who should see it. The photograph flowed along relationships according to user choices, the same way the chess game's moves flowed between the two players, and it landed on each recipient's device as a local copy they hold and control. Now the queries that no existing layer can answer become index lookups:

- "Everything depicting Alice" finds `DEPICTS` frames where `(VALUE) = Alice` — photographs, drawings, movies, anything with a `DEPICTS` frame referencing Alice.
- "Photographs of Alice specifically" narrows further by the item's archetype: only items whose manifest declares `PHOTOGRAPH`.
- "All graduation photos" finds `OCCASION` frames where `(VALUE)` = graduation.
- "Photos taken near this location" finds `PLACE` frames whose `(LOCATION)` coordinates fall within a radius.
- "Photos taken by this device in June 2024" filters `CAPTURED` frames by `INSTRUMENT` and `TIME`.

The frame query and the archetype constraint compose naturally: the frame index finds items by their semantic content, the manifest identifies what kind of item each is, and the two intersect. No crawling. No NLP. No reconstruction. The meaning was captured when the photograph was created, by the person who knew what the photograph was of.

The archetype `PHOTOGRAPH` declares that photographs are expected to carry `DEPICTS`, `PLACE`, and `CAPTURED` frames. The accumulation surface is open regardless. Later, someone else adds:

```
LIKE      { (THEME) = the-photo, (AGENT) = Carol, (VALUE) = “Great shot!” }
FUNNY     { (THEME) = the-photo, (AGENT) = Jeff, (VALUE) = “😂”}
RECOMMEND { (THEME) = the-photo, (AGENT) = Dave, (RECIPIENT) = Eve }
DEPICTS   { (THEME) = the-photo, (VALUE) = sunset }
```

Each is a first-class assertion with its own predicate and roles, not a flat comment or tag. Carol’s `LIKE` travels back to the photographer along the same relationship the photograph traveled to Carol. Dave’s `RECOMMEND` reaches Eve directly, because the frame binds her as `RECIPIENT`; the photographer need not be involved. The photograph accumulates reactions, recommendations, and additional descriptions the same way a chess game accumulates moves: signed assertions from identified parties, cohering around the same anchor, without a platform brokering any of it. The `PHOTOGRAPH` archetype did not mention `LIKE`, `FUNNY`, `RECOMMEND`, or a third-party `DEPICTS`. It did not need to.

A Spanish speaker sees the same photograph through Spanish lexemes. The frames are the same; the words that surface them differ. No translation has occurred. The meanings were language-neutral from the start.

The architecture closes a circle: even sememes themselves would be items. The sememe `METER` carries a `GLOSS` frame in English, a `GLOSS` in Spanish, a `DIMENSION` frame (`LENGTH`), `CONVERSION` frames to other units, and a `SYMBOL` frame (“m”). In each language’s lexicon, a lexeme points at this sememe: “meter” in English, “metro” in Spanish, “メートル” in Japanese. The meaning is not a definition string but the structured totality of everything asserted about it. A language itself (English, Spanish, Japanese) is an item whose frames include its entire lexicon. The vocabulary lives *in* the graph, as items made of frames, using the same primitives as everything else. Your local runtime carries the vocabulary you have encountered; when you meet a sememe you have not seen, you fetch it from peers who have it, the same way you fetch any other item.

The comparison with files, made explicit:

Files	Items
Opaque bytes; no layer interprets content	Typed frames; the layer knows what everything means
Named by path in a hierarchy	Named by content-addressed IID; discoverable by meaning
No built-in authorship, versioning, or integrity	Every frame is signed; every version is a manifest hash
Metadata is a sidecar (EXIF, xattr, .DS_Store)	Metadata IS frames, first-class and queryable
Relatedness means same folder or a hyperlink	Relatedness is structural: frames bound to the same anchor
Application decides how to interpret it	Frames describe the item in the shared vocabulary; any application can read them
Search by filename or keyword	Query by meaning across the graph
Lives on a server or a single device	Lives on your device, replicated to chosen peers
Disappears when the service shuts down	Persists as long as any peer holds it

The item is what would replace the file for the user. Not at the POSIX level (bytes and streams are a fine substrate for low-level I/O) but for user-facing data: the things people create, name, share, organize, search for, and care about. The item is interpretable by any runtime that holds the shared vocabulary, because it is described by frames, and frames are meaning.

7. The Open Commons

The vocabulary described in previous sections provides shared meanings for predicates, roles, and archetypes. Two questions remain: how does the vocabulary handle specific entities rather than general concepts, and how does it grow?

The entity problem

The `AUTHORED` example: predicate `AUTHORED, (THEME) = The Hobbit, (AGENT) = Tolkien`. `AUTHORED` is a shared meaning. `PERSON` is a shared meaning. `BOOK` is a shared meaning. What about Tolkien *himself*?

Today, J.R.R. Tolkien exists as a Wikipedia page, an Amazon author page, a Goodreads entry, a TMDB profile, a Library of Congress authority record, a Wikidata entry, and countless other disconnected representations. None is the canonical Tolkien that every system could use as the `AGENT` binding.

This is the hardest problem the shared meaning space must address. WordNet provides the *concept* `PERSON`, but not a unique identifier for every specific person. Every previous attempt to solve this has faced the same dilemma: centralized registries (Wikidata, Library of Congress) work but create gatekeepers, while fully decentralized naming avoids gatekeepers but introduces ambiguity.

The approach taken here is that entities are items: cryptographic anchors described by frames. Tolkien, in this picture, is not a string or a URL or a row in a registry. He is an item around which frames cohere: frames that assert his name, birth date, works by him, works about him, relationships. These frames are signed by the people and institutions that assert them, and the item endorses whichever of those frames its maintainer accepts.

In practice, multiple items representing the same real-world entity will inevitably exist. Alice creates a Tolkien item for her reading history (though an author this well-known almost certainly already has items maintained by others that she could have referenced instead). The Library of Congress has one for its authority records. A university library has another. Each was created independently, each has a different IID, and each is described by frames from a different angle.

Convergence happens socially. Alice encounters the Library of Congress's Tolkien item through a peer and recognizes it as the same person her item represents. The natural resolution is merge: the frames from both items converge around one canonical IID, likely whichever is most widely referenced or most trusted. Which IID wins is not decided by the substrate but by the community of peers who reference it.

Merge may not always happen, and that is acceptable. When two communities genuinely disagree about whether two items represent the same entity, or when conflicting frames make a clean merge impossible, multiple items coexist. A `SAME_AS` frame can assert that two items represent the same entity, but the assertion is a signed claim by whoever makes it, not a system-level fact. Different users may accept or reject it based on their own judgment. Honest disagreement, represented as coexisting items with overlapping but conflicting descriptions, is what decentralized identity looks like when no authority can declare a winner.

I will not pretend this is a solved problem. It trades the problems of centralized identity (political control, single points of failure) for different problems (convergence latency, conflicting representations). I believe the trade-off is correct, but the entity problem remains the area where the architecture is most genuinely unproven.

How the vocabulary grows

The shared meaning space is not a closed vocabulary. Domain-specific communities can extend it with their own concepts (medical terminology, legal concepts, engineering standards), connected to the base through the same hierarchical relationships. New languages connect by linking their words to existing meanings. The vocabulary grows from the edges, not from the center: extensible without fragmentation, because every extension is anchored in the shared backbone. Extensions propagate the same way every other item does, through signed, content-addressed replication between trusting peers. A medical community defines its vocabulary among its own peers without permission from a central authority; anyone outside who wants access fetches it through the same substrate. The commons has no curator, only authors and relationships.

8. The Trust Matrix

Every frame carries meaning in its predicate. A `FUNNY` frame is not a generic "reaction"; it is a specific semantic assertion that its target is funny. `HILARIOUS` and `AMUSING` cluster naturally under a common ancestor in the vocabulary hierarchy (something like `HUMOR`); `SPAM`, `ASTROTURF`, and `JUNK` cluster under another. These clusters are not engineered categories; they fall out of the vocabulary's existing structure. And because frames can target other frames, assessments compose: a `SPAM` frame whose `THEME` points at another frame means "I assert this specific assertion is spam." Assessments of assertions are themselves assertions, signed by identified parties, subject to the same evaluation as everything else.

Trust is what emerges from the accumulation of these assessments. If Alice consistently reacts to Bob's content with `FUNNY`, `INSIGHTFUL`, or `AGREE`, her Librarian computes a trust in Bob's judgment in those domains from the pattern alone. If Carol's Librarian has been reliably relaying Alice's messages, Alice's Librarian computes infrastructure trust from its operational history. The trust is the pattern; the pattern is the data. Explicit trust declarations are possible but rarely necessary; most trust accumulates naturally from interactions the system already records.

The result is not a single number but a **matrix**: multi-dimensional, with separate assessments for separate domains. I might trust Alice's taste in music without trusting her political judgment, or trust Bob's Librarian for reliable relay without trusting Bob's content. Even identity verification (how confident I am that this key represents the person I think it does) is one dimension among many, extending PGP's coarse, single-axis web-of-trust into a multi-dimensional, domain-aware structure.

Moderation falls out of the matrix without a separate system. I mark several of your posts as `SPAM`; my trust in your content decreases. Frank disagrees and marks my `SPAM` frames with `DISAGREE` (frames targeting frames). Others may add their own frames targeting any of these with their own opinions (even a whole discussion thread may result, made up of these frames themselves). For users who trust my moderation over Frank's, the weight of accumulated `SPAM` assertions reduces your posts' visibility, increasingly so as similar assertions accrue from others they trust; for users who trust Frank's judgment over mine in this domain, your posts appear unaffected. No moderator was appointed, no appeals board convened, and no single outcome was imposed. Two domains work simultaneously: your trustworthiness as a content creator, and my trustworthiness as a moderator.

Trust is also **transitive**, and the strongest form requires no explicit endorsement. If Alice, Bob, and I independently react positively to the same restaurants, our Librarians compute an overlap in taste from the convergence of independent assertions about the same targets. This convergent-taste signal is more reliable than explicit endorsement, because it cannot be faked without faking the underlying reactions themselves. Where direct convergence is unavailable (perhaps I haven't seen enough of Bob's behavior to build trust in him myself), an explicit trust query works. I send `TRUST { (THEME) = Bob, (ATTRIBUTE) = FOOD }` to Alice's Librarian, a peer whose judgment I already trust. Her Librarian evaluates the query against its computed matrix and returns her assessment of Bob's taste in food. I now have Alice's domain-specific opinion of Bob, weighted by my trust in Alice.

The structural shape of this should be familiar to anyone who has worked with recommender systems or machine learning. A trust matrix is a per-user weighting function: inputs are the signed assertions a Librarian has seen, outputs are modulations of visibility, ranking, and routing, and weights are learned from observed patterns rather than declared up front. Collaborative filtering (Goldberg et al., 1992; Resnick et al., 1994) — the core technique behind most modern recommendation engines — is the foundation: if many people I trust agree on something, I likely will too. Decades of research into collaborative filtering and graph-based recommendation have worked out how such computations behave at scale, and the trust matrix inherits that lineage directly. What distinguishes it is not the mathematics but where it runs and whose interests it serves. Weights name specific people and specific domains rather than features of a black-box model, so any of them can be inspected, explained, or overridden. It is tuned to match the user's own behavior and preferences, not an operator's revenue or engagement target. A recommender tuned to platform metrics becomes an instrument of user manipulation; the same mathematics placed under user control becomes a tool of user agency.

Each Librarian computes its own trust matrix **locally**. There is no global trust score, no universal reputation number, no platform-computed ranking. The user always retains the ability to override computed values, and the trust algorithms themselves are implementations — items in the graph — that can be replaced with alternatives. Two users looking at the same content may see different things because their trust matrices differ. Golbeck (2005) argued that trust is fundamentally personal — that "calculations about trust must be made from the perspective of the individual" — a principle the architecture here implements not as a personalization layer over shared content, but as the organizing structure of the substrate itself.

A trust model that requires relationships to function raises an obvious bootstrapping question: how does a new user get started? The answer is the same as in any social system. You arrive through someone you already know, a friend who peers with you, a community node that accepts newcomers, or a public gateway that offers initial connectivity. Trust starts small and grows through interaction. This is not a limitation unique to the proposed architecture; it is how human social networks have always worked. You do not arrive in a new city knowing everyone; you start with a few contacts and expand.

9. Computation as Frames

For frames to serve as the primitive of a substrate rather than a metadata system, they must be expressive enough to handle domains far removed from natural language. Mathematics is a strong test case: the most formal, least ambiguous domain of structured knowledge. If thematic roles can describe mathematical operations, they are not linguistic conveniences, but rather universal structuring principles.

Arithmetic

Take the expression: `3 + 5 = 8`. The operation `ADD` is the predicate. The operands are not Agents (they don't initiate anything) or Patients (they don't change). Addition is structurally a transfer: one quantity is delivered to another. The 5 is the Theme (the quantity in motion); the 3 is the Goal (the destination). Natural language preserves this directly: we say "add 5 *to* 3," not "add 3 and 5 symmetrically."

```
ADD { (THEME) = 5, (GOAL) = 3 }
```

The frame is the input form: a predicate and its bindings, nothing more. Evaluating the frame produces a value, in this case 8. That value plays the role of Result in the cognitive structure (the thing that comes into existence through the operation), but it is not a binding on the input frame. It is what comes out the other end when the frame is run against an implementation of `ADD`'s contract.

Subtraction takes the same transfer structure, reversed: `SUBTRACT { (THEME) = 3, (SOURCE) = 10 }` evaluates to 7. The 3 is the Theme; the 10 is the Source (where it is removed from).

Multiplication recruits a different role entirely: `MULTIPLY { (THEME) = 3, (INSTRUMENT) = 5 }` evaluates to 15. We say "multiply 3 *by* 5". Addition and subtraction are transfers, with Source or Goal endpoints; multiplication is a modification, with an Instrument that scales the Theme. Different operations recruit different roles, but the same small inventory describes them all. The roles were defined for natural language; they apply to math because the cognitive operations underneath are the same.

Calculus

The definite integral of x^2 from 0 to 1:

```
INTEGRATE { (THEME) = x², (SOURCE) = 0, (GOAL) = 1, (INSTRUMENT) = dx }
```

Source and Goal for the bounds of integration. These roles were defined for physical motion ("move from the house to the store"). The application here is metaphorical — integration is not motion in the literal sense — but mathematics is abstract by nature, and the cognitive structure carries: a starting point, an ending point, a sweep across the interval. Evaluating the frame produces ⅓, the Result.

Differentiation and limits follow the same pattern: `DIFFERENTIATE { (THEME) = x², (INSTRUMENT) = x }` evaluates to 2x; `LIMIT { (THEME) = 1/x, (GOAL) = ∞ }` evaluates to 0, the variable approaching the Goal the same way a physical object approaches a destination.

The `x` in these examples is a literal key element, as described earlier: a user-defined name rather than a vocabulary meaning. Variable binding (`x = 5`, or `EQUALS { (THEME, x) = 5 }`) uses a literal in the compound key to name the variable. This is what allows the frame model to express computation with variables, not just fixed values.

The role mapping

The pattern across both arithmetic and calculus reduces to a small mapping between math concepts and thematic roles:

Math concept	Thematic role	Linguistic parallel
Operand / expression being operated on	Theme	"the thing being acted on"
Origin / starting value / what is subtracted from	Source	"where from"
Destination / ending value / what is added to	Goal	"where to"
Scaling factor / variable / differential	Instrument	"by what means"
Degree or magnitude of change	Extent	"by how much"
Path of integration (line integrals)	Path	"the route taken"
Answer / output	Result[2]	"what comes into existence"

The mapping is significant not because it enables a math engine (though it does: `5 meters + 3 feet` is an `ADD` frame whose operands are quantities with unit sememes, resolvable because `METER` and `FOOT` are both `LENGTH` units with known conversion factors). It is significant because it demonstrates that thematic roles are cognitive structuring principles, not linguistic artifacts.

Mathematics and natural language both express: what is being operated on (Theme), by what means (Instrument), where we start (Source), where we end (Goal), by how much (Extent), and what results (Result). The roles are the same because the underlying cognitive operations are the same.

If the same small set of thematic roles can structure natural language, social interactions, and mathematical expressions, those roles are genuinely universal. A base layer built on them is as general as meaning itself.

Mathematics as a language

As noted earlier, predicates may carry parsing behavior: `+` is a token whose corresponding sememe `ADD` declares itself as infix with a precedence and associativity. The consequence for mathematics specifically is that natural language, mathematical expressions, and domain-specific notations coexist within a single input stream. "find books authored by Tolkien where price < 20 dollars" mixes English with a mathematical comparison, and both resolve through the same pipeline. "Show games where Fischer opened 1. e4 e5 2. Nf3 and won in under 30 moves" mixes English with chess algebraic notation and a numeric filter. "5 feet + 30 cm in meters" mixes unit-bearing arithmetic across two measurement systems. In each case, every token resolves to a sememe, every

2 Result here names the role for the output of evaluation, not an input binding on the frame. The other rows describe input bindings.

sememe declares its parsing behavior, and the language being spoken is inferred from the tokens rather than assumed.

Mathematical expressions are not bolted onto the side of a semantic layer. They are frames. A spreadsheet cell is a frame whose value is the result of an expression frame and a spreadsheet itself is just an item with such frames. The boundary between "data" and "computation" dissolves the same way "data" and "metadata" does: both are role bindings on predicates.

The contract and the code

A predicate carries a *contract*: it declares what values it expects, what result it produces, and how it might be parsed and evaluated. The contract, however, is not the code that satisfies it. `ADD` can declare that it takes two operands and produces a sum; it cannot, by itself, compute the sum.

The answer that fits the rest of the architecture is that code is published the same way every other thing is: as items. An implementation of `ADD` would itself be an item, carrying the executable form (source code, compiled bytes, or a formal specification) alongside frames declaring which contract it satisfies, who signed it, and what runtime is needed. The relationship between an implementation and the predicate it satisfies is itself a frame.

A predicate could have many implementations: different runtimes, different trade-offs, different authors, all coexisting. A runtime evaluating an `ADD` frame would pick an implementation it can execute and whose author it trusts. The contract is the meaning; the code is the machinery. Nobody would own the contract, because `ADD` is a sememe in the shared vocabulary, no different from `BOOK` or `AUTHORED`. Anyone could publish an implementation item, sign it, and let it propagate; whether a particular runtime uses it would be a matter of local trust and user choice, not central authorization.

Code distribution would become a specific case of data distribution. Today, software reaches users through centralized clearinghouses: app stores, package managers, vendor release servers. In the picture being described, code would travel the same peer-to-peer mechanism as any other item: signed, content-addressed, replicated through relationships, versioned, forkable. The package manager dissolves into the same medium that carries the rest of the data.

The choice raises serious security concerns. Running code from arbitrary peers is a recipe for disaster unless the runtimes loading and executing it are properly sandboxed. Sandboxing in this picture would not be a separate special system; it would be another kind of policy attached to frames, the same way replication or retention policy would be. The problem is hard, but it is the same kind of hard as sandboxing untrusted JavaScript in a web browser, and the existing landscape of techniques (capability-based interfaces, isolated execution environments, formal verification of restricted languages) gives plenty to draw on.

An obvious objection is that social trust is insufficient for authorizing code execution. The objection deserves a precise answer: *all code distribution already relies on social trust*. When you install software from the Google Play Store, you are trusting Google's review process. When you install a package from apt, you are trusting the Debian maintainers. When you download from the Apple App Store, you are trusting Apple. All of this is still "social" in a very real, society-wide sense. Even reproducible builds only verify that the build is reproducible, not that the code is safe; someone still has to audit it, and you still have to trust whoever did. Supply-chain attacks like SolarWinds and the xz backdoor demonstrate that centralized trust intermediaries are not immune to compromise.

What this proposal changes is not whether code execution depends on trust, but who is being trusted: a personally-chosen network of peers whose reputations are visible and whose endorsements are signed, rather than an opaque corporate process whose incentives may not align with your safety. Nothing would prevent Google or Apple from publishing their own code items in the graph, complete with signed reviews and recommendations of other applications. Users who trust their judgment can continue to rely on it. The difference is that Google must stand alongside every other reviewer rather than occupying a privileged gatekeeping position that no one else can fill.

Because code is an item and data is an item, one of the main structural rationales for SaaS centralization dissolves. Most software became a service in part because the operator held both the code and the data, and running the code required their infrastructure. Here, both travel through the same peer substrate, and execution

happens wherever the user has a runtime, most naturally on their own device. The server farm, the subscription, the operator as gatekeeper: these arrangements lose the structural basis that made them seem like the only option. Computation needs no new primitives. The contract is a sememe played as a predicate; the code is an item; and the link between them is a frame — all structures the substrate already provides.

10. Sanity Check

The architecture is ambitious: every assertion stores meaning, every frame is signed, every semantic binding is indexed, and the whole substrate runs peer-to-peer without central servers. Reasonable readers will have questions. Three stand out: does it scale, what does the attack surface look like, and how do users actually come to hold the keys this requires? All three deserve direct answers.

Scaling

Scaling in a peer-to-peer substrate is a different question than scaling a centralized service. No single server's capacity caps the network; each node carries its own indexing burden, growing with the assertions it has reason to track. The substantive questions are how that per-node cost behaves under each pressure: the overhead frames and items add to raw content, how indexing cost grows under engagement and distributes across the network, what language and text impose, and what the cumulative figures look like at the scale of complete real-world databases.

Frame overhead

A frame has two parts: a **body** carrying the semantic assertion (a predicate and its bindings) and a **record** attesting it (body hash, signer key ID, timestamp, Ed25519 signature). Both are content-addressed and encoded in a deterministic profile of CBOR (Concise Binary Object Representation; Bormann & Hoffman, 2020), a binary format more compact than text-based serializations like JSON. All identifiers are 34-byte multihashes (a 2-byte type prefix followed by a 32-byte SHA-256 hash).

A simple body (predicate plus a pair of bindings, no text payload) runs ≈200 bytes, which is effectively the minimum. More intricate assertions carry more bindings and are correspondingly larger, but each binding adds only a small amount. The record envelope starts at ≈126 bytes.

Indexing and aggregation

Every frame creates entries in a **fan-out index** for each binding whose target is an item or another frame, enabling reverse lookup from any participant: 136 bytes per entry (102-byte key, 34-byte reference). A second **record index** maps signers to the frames they attested, at 102 bytes per entry. Literal payloads and policy bindings are not indexed.

Indexing cost grows linearly with the number of frames, not with content size. A photograph with six descriptive frames creates about ten index entries regardless of whether the image is a megabyte or a gigabyte. A user with 10,000 photographs, 500 books, and 1,000 social posts would hold roughly 160,000 entries, about 22 megabytes.

The question is what happens at the extremes. Social media engagement follows a steep power-law distribution. Buffer's analysis of more than 52 million posts (Buffer, 2026) found median engagement in the low single digits per post on text-centric networks like Bluesky and Mastodon, with median engagement rates between 2.5% and 6.2% on larger platforms. A typical post generates a kilobyte or less of index data; a popular post with ten thousand reactions, roughly three megabytes. The all-time record is 74 million likes on a single Instagram post.

For the extreme case, consider a viral post with five million reactions. 70% are simple reactions with no text (200-byte bodies), 25% carry short comments (350 bytes), 4% longer comments (700 bytes), and 1%

substantial text (2,500 bytes). Add second-order reactions (replies to comments, roughly 500,000 additional frames) and the total is roughly 5.5 million frames.

Component	Calculation	Size
Frame bodies (semantic content)	5.5M frames × 290 bytes (weighted average)	1.6 GB
Frame records (attribution envelopes)	5.5M × 126 bytes	690 MB
Fan-out index	5.5M × 2 entries × 136 bytes	1.5 GB
Record index	5.5M × 102 bytes	561 MB
Total		**≈4.3 GB**

Approximately 4.3 gigabytes for the most extreme case in the distribution. That is the total across the entire network. The peer-to-peer model distributes this cost. In a centralized system, one server cluster holds every reaction to a viral post. In this architecture, no single node needs to hold all the reactions, though any node that wants to can reach across the graph to accumulate them. Each Librarian's index reflects its own social reach.

Node type	Reactions held	Total storage
Casual viewer (few hundred peers)	50-200	40-160 KB
Active participant (few thousand peers)	2,000-5,000	1.5-4 MB
Popular creator (500K followers, 10% react)	50,000	40 MB
Community hub or relay node	100K-500K	80-400 MB
Analytics company (wide social reach)	1-3M	1-2.5 GB
Full aggregator (near-complete)	5M+	4.3 GB

The social graph is a natural shard boundary, and the per-node cost is bounded by that node's actual relationships.

Signing is not a bottleneck. Ed25519 (Bernstein et al., 2012) performs at least 50,000 signatures per second and 15,000 verifications per second on commodity hardware. The five-million-reaction extreme case is roughly 100 seconds of signing work distributed across millions of users, and about five minutes of sequential verification for a single node that wanted to check every signature.

The distributed model raises its own question: how do you count reactions you do not hold? If no single node has seen every reaction to a viral post, how does anyone report "1.2 million likes"? This problem only arises for items that accumulate reactions beyond what any single node's social graph delivers, a small fraction of all items given the power-law distribution described above, so the vast majority of items never need aggregation at all. For those that do, the solution comes from distributed cardinality estimation. HyperLogLog (Flajolet, Fusy, Gandouet, & Meunier, 2007) is a near-optimal probabilistic algorithm for exactly this problem, widely deployed in production systems (Redis, PostgreSQL, BigQuery, and others). Each node maintains a compact probabilistic sketch (a HyperLogLog register, roughly 16 kilobytes regardless of the underlying count) that summarizes the reactions it has seen. These sketches are mergeable: combining sketches from multiple peers produces an estimate that correctly deduplicates reactions seen by more than one peer, even when social graphs overlap arbitrarily. The merged result has bounded error, typically within two percent. The sketch itself is just another frame, signed and replicated through the same trust graph as everything else.

Language and text indexing

Text handling raises a separate scaling concern. The token dictionary holds two fundamentally different kinds of text. The first is **vocabulary**: lexemes from language imports, mapping words to their meanings. This component is effectively bounded. A language has a finite number of words, cataloged in resources like WordNet, and the cost of importing a language is fixed and known in advance. Languages do grow over time,

and users can coin arbitrary terms, but the rate of vocabulary growth is trivial relative to the base. The second is **user content**: titles, proper names, and other text from `VALUE` bindings that users create over time. This component is unbounded, growing linearly with the items indexed. The two have different scaling profiles and are worth examining separately.

The resolution pipeline that serves both uses a lattice of overlapping candidate spans (single words, two-word windows, three-word windows) scored by a Viterbi path (Viterbi, 1967) to find the best interpretation. Multi-word phrases ("the shawshank redemption") are indexed as single tokens alongside their individual content words, so that both exact-match and partial-match queries work. This windowed approach, designed for multi-word expressions in English, turns out to be exactly the algorithm that Chinese and Japanese word segmentation requires: overlapping character windows resolved against a dictionary. One tokenizer, all languages.

The vocabulary component is bounded by the linguistic resources it draws from. Loading a language has two parts: the **token dictionary** (the lookup index from text to sememe references) and the **semantic graph** (each sememe's frames carrying its hypernym, hyponym, meronym, antonym, similar-to, gloss, example, and source-identifier relationships).

The token dictionary cost per language:

Language	Source synsets	Lexeme entries (with inflections)	Token dictionary size
English (OEWN)	≈108,000	600-700K (modest inflection)	80 MB
German (OdeNet)	≈36,000	800K-1.2M (highly inflected)	190 MB
Mandarin (COW)	≈42,000	≈62K (uninflected; one form per lemma)	4 MB
Japanese (JWN)	≈57,000	≈200K (many verb and adjective forms)	55 MB
Typical language	30-80K	200-500K (varies with morphology)	10-130 MB

Source synset counts are per-language WordNet sizes; through CILI, most concepts overlap across languages, so loading multiple languages does not multiply unique-sememe counts linearly. Lexemes and token dictionary size, by contrast, are additive: each language adds its own word forms pointing at the (mostly shared) sememes.

The semantic graph adds significant additional cost on top of the token dictionary. At an average of 11 frames per sememe (relationships, glosses, examples, source identifiers) with full overhead (200-byte bodies, 126-byte records, fan-out and record index entries, manifest contribution), each sememe item costs roughly 9 KB. For English (108K sememes), the full semantic graph is roughly 970 MB; including the token dictionary, the complete English vocabulary is roughly 1 GB.

A polyglot user loading five languages would have a complete vocabulary of roughly 2 gigabytes. Loading every language for which resources exist (100+) would reach about 9 gigabytes, with the marginal cost per language dropping as smaller languages are added. Both are routine on commodity hardware.

User-generated content (titles, proper names) grows linearly with the items indexed. Each token dictionary entry carries 41 bytes of fixed overhead (content hash, binding index, weight) plus the token text itself, so entries range from under 50 bytes for short words to over 100 for long multi-word titles. A node carrying ten million titles (the approximate scale of IMDB) averaging four indexed words each would add roughly 40 million entries, on the order of two gigabytes. Function words (articles, prepositions, conjunctions) that carry no discriminating value for search can be excluded from individual-word indexing at the direction of the language item itself, which participates in parsing and knows which of its word categories are worth indexing. This reduces the entry count by roughly 30-40%, since function words are disproportionately common in titles.

A complete database example

A worked example puts these numbers in perspective. Consider a complete film database at the scale of IMDB — the structured semantic content only. Long-form content (full synopses, biographies, user reviews, photographs, posters, trailers) is addressed by hash and stored separately; that cost is identical in any

architecture and so is excluded from this analysis. The semantic structure consists of roughly 10 million titles (movies, television series, episodes, shorts) and roughly 12 million person entries (actors, directors, writers, producers, crew). Each title carries frames for title strings, genres, release dates, ratings, and runtimes, plus the relationships that make a film database useful: an average of 12 cast members and 10 crew members per title, each a signed frame binding the person to the title in a specific role. Each person entry carries frames for name, birth/death dates, birthplace, and profession. The full accounting:

Component	Calculation	Size
Title frames (titles, genres, dates, ratings, runtimes)	10M titles × 10 frames × 200 bytes	20 GB
Person frames (names, dates, birthplaces, etc.)	12M people × 5 frames × 200 bytes	12 GB
Cast and crew relationship frames	10M titles × 22 people × 200 bytes	44 GB
Attribution records (all frames)	380M × 126 bytes	48 GB
Item manifests	22M items × 2 KB (estimated average)	45 GB
Fan-out index	380M × 2 entries × 136 bytes	103 GB
Record index	380M × 102 bytes	39 GB
Token dictionary (titles + person names)	65M entries	3 GB
Total for the entire database		**≈314 GB**

Approximately 314 gigabytes for the entire semantic content of a major publicly accessible structured database — every assertion signed, every relationship queryable by meaning. That is a single commodity hard drive. And as with the viral-post example, the cost falls only where the data is wanted: a casual movie fan who has looked up a few hundred films holds a few tens of megabytes, while only the institutional node that maintains the authoritative catalog holds the full volume.

In a typical relational database (PostgreSQL, MySQL), an equivalent dataset with conventional indexing would occupy roughly 53 GB. The substrate's 314 GB is roughly 6× larger, in exchange for architectural properties the relational version cannot match. The breakdown separates indexing-to-indexing comparison from data-side overhead:

Component	Relational DB	Frame substrate
Data (rows / frames)	≈30 GB	≈76 GB
Cryptographic signing envelopes (per-record attribution)	not present	≈48 GB
Item manifests (providing versioning)	not present	≈45 GB
Standard query indexes	≈20 GB	(subsumed)
Text indexing (names only)	≈3 GB	≈3 GB
Universal reverse-lookup (any participant → any frame)	not provided	≈103 GB
Provenance lookup (signer → frame)	not present	≈39 GB
Total	**≈53 GB**	**≈314 GB**

The proposed substrate spends ≈261 GB beyond the relational architecture, in five categories. On the **data side**: ≈46 GB for **frame structure overhead** (each assertion self-describes its own roles rather than relying on a fixed table schema, which is what allows third parties to add new kinds of frames about existing items without coordination), ≈48 GB for **records** (the signing envelopes that make every assertion individually verifiable by its author's key), and ≈45 GB for **item manifests** (signed endorsement lists that record which frames each item officially includes, providing item-level versioning and integrity). On the **indexing side**: ≈83 GB for **universal**

reverse-lookup beyond the bespoke query indexes a relational architecture already requires (any participant in any frame is queryable as the target of every assertion that names it; an RDB matches this only by adding bespoke indexes per relationship type, with costs that grow as queries multiply), and ≈39 GB for **provenance lookup** (signer → frame, enabling "who attested this?" queries).

The 48 GB record figure is a conservative upper bound: in practice, many frames would be endorsed via signed item manifests rather than individually signed, which would reduce the actual attribution overhead substantially below this number.

The 6× total is the price of cryptographic provenance, universal queryability, decomposable structure, and item-level versioning together. On commodity hardware in 2026, both totals fit comfortably on a (not even particularly large) single drive.

The scaling picture is, on balance, comfortable. What makes it comfortable is not just that the absolute numbers are small, but that those numbers buy substantially more functionality than the architectures they replace. The substrate's storage cost is several times that of a relational equivalent, but in exchange every assertion is cryptographically attributable, every relationship is reverse-queryable without per-query index design, and every item is a portable, application-agnostic object that any runtime can interpret. Index growth is linear in the number of semantic assertions, the social graph bounds per-node cost, probabilistic aggregation handles the counting problem, text search scales with content, and signing is fast. No component exhibits superlinear growth.

As this entire architecture is proposed and not yet implemented, all of these figures are entirely estimated, though chosen as conservatively as possible. They are also uncompressed: in practice, key-prefix compression, block compression, and other storage optimizations would bring these totals down significantly.

Attack surface

The absence of a central server changes the economics of attack. In a centralized system, a distributed denial-of-service (DDoS) attack works because there is a single point of failure: overwhelm one target and the service goes down for everyone. This architecture has no such target. Each user runs an independent Librarian, and taking one down affects one user; everyone else is unaffected. Larger institutional nodes (a major catalog provider, a community hub) are more visible targets, and an attack on one could be noticed. But because their data has already been replicated across the peers who accessed it, the impact is attenuated: some users might notice degraded freshness, but most will continue operating against local copies without interruption. The time-to-impact window stretches from seconds (in the centralized case) to hours or longer, depending on how widely the data has propagated.

The social graph doubles as an access-control boundary. A Librarian accepts connections from peers it has trust relationships with. An unknown party with no trust path cannot connect, let alone flood. This is not a firewall rule that must be maintained; it is a structural property of how the network is organized. Several attack patterns that threaten conventional peer-to-peer systems are naturally mitigated.

Sybil attacks (creating masses of fake identities to overwhelm a network) depend on new identities being granted influence by default. In most hash-distance-based systems, a new node is assigned a position in the network and immediately participates in routing. Here, a new identity starts with zero trust and must build relationships through sustained, assessable behavior. Mass-creating identities does not help because none of them have trust, and trust cannot be manufactured without the interactions that produce it.

Eclipse attacks (surrounding a target with attacker-controlled nodes to isolate it) require the attacker to control the target's peers. In systems where peers are assigned by network topology or hash distance, this is achievable by positioning nodes strategically. Here, peers are chosen through deliberate trust relationships, not assigned by algorithm. An attacker would need to compromise the target's actual social relationships, which is a fundamentally different and harder attack vector.

Frame flooding through compromised trust paths is the closest analog to conventional DDoS. An attacker who has a trust relationship with you (or a path through mutual peers) could send large volumes of junk frames. Every frame, however, is signed: the source is immediately and cryptographically attributable. The

trust matrix degrades the attacker's score as junk accumulates, and the Librarian stops accepting frames from them. The attack is self-limiting because it burns the trust relationship it depends on.

Content poisoning (publishing frames with false or malicious content) is attributable for the same reason. The liar is identified by their signature, and content-addressing means data cannot be tampered with in transit since the recipient verifies the hash locally. The trust matrix adjusts, and other peers who trust the recipient's moderation judgment see the adjustment propagate.

Network-layer attacks remain possible. The substrate operates above the transport layer and does not change the physics of TCP/IP. An attacker can still saturate a specific machine's bandwidth. The consequence, however, is limited to that machine. Because every item a user has previously accessed is persisted locally (not cached ephemerally, but held as a durable local copy), even taking a popular node offline for hours may go unnoticed by users who already hold the data they need.

The attack surface that remains is **key compromise**: if an attacker obtains a user's private key, they can sign frames as that user. No purely software-based substrate can fully address this, because the threat model includes physical access to the device where the key is stored.

The fundamental shift is economic. Generating network traffic is cheap; building fake social trust is expensive, slow, and self-defeating. Every attack that depends on trust must first earn it, and every abuse of trust is signed, attributable, and self-correcting through the trust matrix. The architecture does not claim to eliminate attacks, but it changes what is required to mount one and ensures that the cost of attacking scales with the trust the attacker must build and then burn.

Custody and transition

Users have been conditioned over decades to offload identity management to platforms. "Forgot password?" flows, federated logins, and cloud-hosted credentials train users to treat their accounts as things a service holds on their behalf. The substrate's promise reverses this: the keypair belongs to the user, and no central authority can grant or revoke it. Architecturally this is a strength; practically it is an adoption challenge, because users who have never managed their own keys cannot reasonably be asked to start doing so overnight under threat of losing their entire history if they fail. The question is not whether the architecture supports self-sovereign keys — it does — but how users arrive at self-custody from where they currently are.

The answer is a spectrum of custody, not a single model. The substrate supports every level on it; users migrate along it as comfort with key responsibility grows.

1. **Gateway-custodial.** A bank, email provider, or access gateway holds keys on the user's behalf — functionally similar to today's custodial relationship with platforms, but with data in the substrate's semantics, so it remains portable if the user later migrates. This is where most users will enter: onboarding costs no more than creating any other account, and from day one the user's data is substrate-native rather than trapped in an application-specific silo.
2. **Device-bound with platform recovery.** Keys live in the device's secure enclave (Apple's Secure Enclave, Windows TPM, Android Keystore, or equivalent) and sync via the platform's credential infrastructure. This is the model that passkeys and platform keychains have already made familiar to mainstream users. The user trusts their platform vendor for recovery, which is roughly the trust level they already have; the substrate's portability means that trust is bounded and revocable rather than total.
3. **Social recovery.** The user's key is split via a threshold scheme (Shamir, 1979) into N shares distributed across trusted peers; any M of them can reconstruct the key if direct access is lost. The approach requires a set of trusted peers to exist, which is precisely what the substrate's trust graph already provides. Threshold secret-sharing has been operational in cryptocurrency wallets for years and transfers directly.
4. **Hardware-based self-custody.** A dedicated device holds the key offline; signing happens on-device over a limited interface. Existing hardware security tokens and cold-storage wallets can serve, though they were built for adjacent purposes (web authentication, cryptocurrency custody) and address device compromise without fully addressing coercion. Keymaster (Chambers, 2025), a sister project, is purpose-built for this role. It is designed to be everyday-carried like a physical key on a keyring,

affordable enough to be standard rather than specialist, and the only hardware answer to both device compromise and coerced disclosure (threats software alone cannot fully address). The same model scales upward. Multiple devices, air-gapped signing, policy-enforced workflows, and multi-party approval support users and organizations with the highest threat models: effectively impossible to compromise, deliberately inconvenient, and suited to high-value assets or sensitive institutional use.

Custodial entry and progressive sovereignty are complementary rather than contradictory. Identity persists across moves up the spectrum; only the custody relationship changes. A user can start at level 1 with their bank holding their keys, move to level 2 when they set up a passkey-equipped device, add level 3 recovery once they have built trust relationships, and ultimately reach level 4. The substrate's portability is what makes migration possible — at each level the keys control the same identity, because identity in the substrate is a keypair, not an account at a service.

Two mechanisms complement the spectrum. Delegated session keys let a primary key authorize per-device session keys that are individually revocable, usable at any level: losing a laptop revokes a session key rather than the primary. Graceful degradation handles the worst case of irrecoverable loss: the user creates a new identity and their community re-endorses them under the new key — "this is the same person, whose previous key is retired" — which is how identity actually works when someone changes a name or phone number. None of these mechanisms are new; what the architecture contributes is a substrate where they compose naturally with the trust graph already in use. The remaining hard case — coerced key disclosure, where the user is physically compelled to sign — is what Keymaster is designed to address, and is the only part of the custody problem that substrate-level software cannot solve on its own.

11. Consequences

The proposed architecture has consequences that extend beyond the technical. Several are worth naming because they are not obvious from the primitives alone.

The most visible change is the **subsumption of platforms**. A product listing, a community forum, a social feed, a review, a citation graph: each of these is currently a row in a proprietary database on a proprietary platform, and each would be expressible as frames in the shared vocabulary. The platform-specific data model that locks users in dissolves, because the data model is shared. Applications no longer own the data they operate on. An email client, a document editor, a social feed, a project tracker would all be applications over frames, and any application that understands the archetype can read the same data. Switching applications does not mean losing your work, because the data is self-describing and the applications are bound by the archetype's contract.

The economics of the internet do not disappear. Businesses still want customers, individuals still want to find information and buy services, and hosting remains relevant for commercial interests that want an always-available presence. What disappears is the ability to monetize user entrapment: the platform business model built on lock-in. The hosting and service business model, where providers compete on quality, reliability, and price rather than on the structural impossibility of leaving, remains entirely viable.

Formalized relationships between parties on public networks are expressible in the same substrate. The range of *smart contracts* described by Szabo (1997), from bearer certificates to multi-party commercial transactions, composes naturally from signed frames exchanged between identified parties. This is arguably a more faithful realization of Szabo's vision than the blockchain-based systems that later popularized the term: contracts between identified parties, cryptographically enforced, without requiring a global shared ledger as the enforcement substrate.

Real-time communication is another consequence rather than a separate system. A phone call is a `CALL` frame with audio in a `VALUE` binding, streamed through the peer network. A video meeting is an item whose participants each contribute `PRESENT` frames with video and audio bindings. The signaling that protocols like WebRTC handle out-of-band (who is calling whom, what codecs, what endpoints) becomes in-band, because the frame IS the signal. Phone numbers become unnecessary when identity is cryptographic and routing follows trust paths. Spam calls become structurally impossible in the default posture: no trust path, no ring. Users who want to accept cold calls (a business, a support line) lower their threshold; the trust matrix still provides a

gradient between “known friend” and “complete stranger” that the spoofable phone number never could. The phone industry’s radio infrastructure remains valuable as transport, but the vast economic superstructure built on top of it (carrier lock-in, number portability battles, robocall mitigation) simplifies dramatically when a call is just a frame routed through peers.

Language models trained on semantic data would be qualitatively different from current ones. LLMs today are trained on flat text corpora in which provenance, trust, and the distinction between direct observation and institutional interpretation are absent. A claim from a peer-reviewed paper, a press release, and a blog post all arrive at the model as undifferentiated training signal, and the resulting model produces statistical consensus from a partially curated mass rather than reasoning that tracks where claims come from. This is an architectural limitation, not a matter of scale. A substrate that preserves signed provenance, trust relationships, and the observation-versus-interpretation distinction at the data level enables something categorically different: models that can natively distinguish who asserted what on what basis, and reason about the structure of disagreement rather than flatten it into an average. The epistemic posture of such a model would be perspective-aware and trust-structured by default, which is a different kind of cognition rather than a marginal improvement in accuracy.

Two properties of the locality pillar deserve explicit mention because they are structural consequences rather than features to be engineered. **Offline capability** is trivial when the runtime and the data are both local; when a network returns, changes propagate to peers. **Resilience to vendor disappearance** is equally structural: items live on users’ devices and on the peers who have replicated them, so a company that shut down or was acquired would lose the ability to ship updates, but the data, the tools, and the peer network would remain. Kleppmann et al. call this “The Long Now” property of local-first software; here it is a consequence of the substrate, not a feature built on top of it.

12. Authorship, Not Ownership

A popular slogan in the decentralization and cryptocurrency communities is “own your data.” The phrase evokes the right sentiment: the lopsided relationship between users and platforms is unjust, something must change, and users deserve agency over their own contributions. The word “ownership” applied to data, however, promises more than any technical system can deliver, and it is worth stating plainly what this proposed architecture does and does not accomplish.

Ownership, in every meaningful sense, implies control. A thing I own is a thing I can exclude others from, a thing I can hold or withhold, a thing that is mine and not yours. These properties hold for physical objects because matter occupies space and cannot be in two places at once. They hold for artificially scarce digital assets like cryptocurrency tokens because the network’s rules enforce the scarcity. They do not hold for ordinary data, and they cannot. Once I have given you a copy, I have no technical means of revoking it. You can keep it, forward it, publish it, transform it, or forget it. My wishes have no technical weight. This is not a failure of the substrate proposed here, or of any honest substrate; it is a property of copyable information, and any system that preserves human agency must accept it.

What the substrate technically delivers is *attribution*: the provable linkage between a signed assertion and the key that signed it. *Authorship* is the broader human concept, the social claim that a particular person created something. Under the usual assumption that the signer is the creator, attribution stands in for authorship closely enough that I will use the words almost interchangeably below, reaching for “authorship” when the human framing matters and “attribution” when the technical mechanism does. The two diverge in edge cases (ghostwriting, delegated signatures, AI-generated content), but those edges do not change the core claim: what the substrate provides is a reliable, verifiable linkage from assertions to keys. Whether the signer is the author is a social-layer question no substrate can answer.

What this proposal can accomplish is closer to the *spirit* of what people mean when they say “own your data,” without claiming the part that cannot be delivered. You hold your own keys, and nobody can sign as you. You author your own assertions, and nobody can forge them. You keep local custody of your data, and no vendor can revoke your access to work you already have. You choose which peers you share with going forward, and you do so through deliberate trust relationships rather than by default exposure to whoever hosts

your platform. These are real, technically enforceable properties. They are not ownership of data. They are ownership of your *participation* in a network: your keys, your authorship, your custody, your consent to new copies.

What the proposal cannot do, nor should any honest proposal claim to do, is compel other parties to honor your wishes about copies they already hold. If you share risqué photographs with a trusted partner and the relationship goes wrong, no substrate can un-share what is already shared. The partial solution is social, not technical: a trust paradigm is what you have when you can *choose* whom to share with, and when the act of sharing is deliberate rather than default. If someone violates that trust, you can stop trusting them, and the trust graph as a whole responds: others who trust you see your revised posture and may update their own relationships with the offender. This is how human communities have always handled the problem. The substrate does not replace that social mechanism; it returns computing to a state where social mechanisms can actually function, because sharing stops being a default and becomes a choice.

Legal frameworks, contracts, and commercial licenses compose naturally with all of this, and the primitives the substrate provides (signed assertions, key-based identity, content-addressed verification) make them easier to encode when they apply. Nothing here displaces them.

The word "ownership" invites confusion because it borrows from property law what the medium cannot enforce. A more honest vocabulary for this project is authorship, custody, and consent. What you can have, in this proposal, is exactly those three: provable authorship of what you assert, local custody of what you hold, and a meaningful say in the propagation of new copies you make. That is substantially more than platforms currently permit, and substantially less than the "ownership" rhetoric suggests. The difference between those two is where honest design lives.

13. Honest Reckoning

I am not the first to propose an ambitious rethinking of how computing handles information. The history of such proposals is rich in cautionary lessons which I would be foolish to ignore.

Predecessors

This list is not exhaustive, but these are the predecessors whose lessons most directly shape this proposal:

Xanadu (Nelson, 1974) envisioned a global, versioned, bidirectional-linking document system. It got content addressing, versioning, and bidirectional links right (concepts that took decades to resurface in Git and IPFS). It failed because it demanded solving everything simultaneously before shipping anything. Lesson: ship incremental function, not a complete vision.

CYC (Lenat, 1995) set out to encode all common-sense knowledge as logical assertions. It got the diagnosis right: computers need world knowledge, not just data. It stalled because hand-authoring millions of axioms does not scale. Lesson: anchor in existing resources and let meaning emerge from use.

Croquet (Smith, Kay, Raab, & Reed, 2003), Alan Kay's vision of shared, replicated 3D environments, got replicated state and seamless collaboration right. It faded because it required a complete runtime (Squeak Smalltalk) and could not interoperate with existing software. Lesson: platforms that cannot meet users where they already are face adoption cliffs no technical elegance can overcome.

Plan 9 (Pike et al., 1995) pushed Unix's "everything is a file" to its logical conclusion. A cleaner and more consistent design than Unix, it nonetheless failed to displace it because it required abandoning the entire Unix ecosystem. Lesson: even a cleaner design loses to an entrenched ecosystem unless it provides a bridge.

The Semantic Web (Berners-Lee et al., 2001) got the diagnosis exactly right: the web needs machine-readable semantics. It built a rigorous stack that works in specialized domains. It did not become general-purpose because it was layered *on top of* the web rather than built into it. Lesson: a semantic layer that is optional will remain marginal.

Local-first software (Kleppmann et al., 2019) is the tradition directly upstream of this paper's second pillar. Its seven ideal properties describe applications that live on the user's device, work offline, sync peer-to-peer, and retain user ownership and control. The tradition is healthy in research and in niche commercial software but has

not displaced the SaaS default, because solving sync and interoperability without a central coordinator, while preserving user experience, has been hard enough that most teams took the easier centralized path. Lesson: the technical problems are now serviceable with current tools; what is missing is a substrate that makes local-first the easier path, not just a more virtuous one.

Lessons

Several patterns run through those predecessors' failures, each worth heeding:

Incremental delivery is non-negotiable. A system that requires completeness before it provides value will never reach completeness. Each increment must be useful on its own.

Build on existing resources. CYC tried to encode all knowledge manually. The Semantic Web required ontology engineering for every domain. This project anchors instead in WordNet, CILI, VerbNet, and ISO 24617-4: resources built and validated over decades by the computational linguistics community. I am not inventing a vocabulary, but rather giving an existing, empirically validated vocabulary a new job.

Provide a bridge. Plan 9 and Croquet demanded that users abandon their ecosystems. A semantic base layer must coexist with files, filesystems, and the web. POSIX is a reasonable base layer for byte handling; it was never intended to be a base layer for meaning. On the web side, this means a gateway: a Librarian that accepts remote clients and renders items as web pages, so that people can discover the system's content through a browser before they install a Librarian of their own. The bridge is how users arrive; the local runtime is where they stay.

Neither pillar can be optional. This is the deepest lesson from the Semantic Web on one side and from local-first retrofits on the other. If creating semantic structure is a separate step from creating data, most people skip it. If local control is a separate feature to opt into, most people stay with the hosted default. The design must make creating data *be* creating semantic, user-held structure, the way writing a sentence *is* expressing meaning, not writing sounds and then separately annotating what they mean.

Can this proposal avoid the fates of its predecessors? Honestly: I do not know. The ambition is large, the history is cautionary, and the engineering challenges are real. The linguistic resources now exist, however. WordNet has ≈108,000 synsets, CILI links them across languages, VerbNet classifies 300 verb classes with role declarations, ISO 24617-4 standardizes the role inventory, and UniMorph (Batsuren et al., 2022) provides morphological data for 100+ languages. These resources represent decades of cumulative scholarly work. They did not exist when CYC began, when the Semantic Web was proposed, or when Croquet was built.

Neither did the technical infrastructure for the locality pillar. Modern signing cryptography (Ed25519 and related primitives) is cheap enough to apply at the per-message level. Content-addressing is ubiquitous (Git and elsewhere), CRDTs have moved from research to shipping practice, and open-source P2P transport stacks (libp2p, modern QUIC implementations) are mature. A local-first substrate in 2026 is not conjuring machinery from nothing; it is composing pieces that have all shipped, some many times over.

And, worth stating plainly: AI assistance has compressed what was previously decades of solo implementation work into feasible timescales, and sustained dialogue with it has contributed to the clarity of this model as a whole. A major bottleneck for ambitious software projects has always been the sheer volume of code required, and that bottleneck has narrowed dramatically. This does not guarantee success, but it changes the economics of ambition.

The path forward is incremental: frames and items as a local data format; a shared vocabulary seeded from semantic and lexical databases; a local runtime that stores, queries, and resolves data by meaning; and a peer-to-peer network where that data is exchanged between nodes connected by trust. Together comprising the semantic and local-first foundation that computing has been missing since the networked era began.

In Closing

Every element in this proposal exists somewhere else. Semantic frames and participant-role theory come from decades of work in linguistics. Grounded, language-independent vocabularies come from the WordNet family of lexical-semantic databases. Content-addressing comes from Merkle trees and their many descendants.

Keypair identity comes from public-key cryptography. Trust as a web of signed endorsements comes from PGP. Distributed state that survives reconciliation comes from CRDT research. The local-first approach has a named tradition and its own articulated ideals. Peer-to-peer networking runs through a lineage from Freenet through BitTorrent, IPFS, Secure Scuttlebutt, and others. The notion that social relationships can organize networks draws on small-world network theory. What this paper proposes is a specific synthesis: which of these pieces belong together, in what relationship, as the substrate of a single layer. The claim of originality is the composition, not any of the components.

Whether it succeeds remains to be seen. I offer it not as a certainty but as a proposal, grounded in established theory and validated resources, that the time is right to try. A reference implementation of this architecture is in active development (Chambers, 2026).

References

Allen, C. (2016). The Path to Self-Sovereign Identity. *Life With Alacrity*, April 26, 2016.

Baker, C. F., Fillmore, C. J., & Lowe, J. B. (1998). The Berkeley FrameNet Project. In *Proceedings of ACL/COLING*, 86-90.

Barnes, J. A. (1954). Class and Committees in a Norwegian Island Parish. *Human Relations*, 7(1), 39-58.

Batsuren, K. et al. (2022). UniMorph 4.0: Universal Morphology. In *Proceedings of the 13th Language Resources and Evaluation Conference (LREC 2022)*, 840-855. ELRA.

Benet, J. (2014). IPFS — Content Addressed, Versioned, P2P File System. ArXiv:1407.3561.

Berners-Lee, T., Hendler, J., & Lassila, O. (2001). The Semantic Web. *Scientific American*, May 2001.

Bernstein, D. J., Duif, N., Lange, T., Schwabe, P., & Yang, B.-Y. (2012). High-Speed High-Security Signatures. *Journal of Cryptographic Engineering*, 2, 77-89.

Bond, F., Vossen, P., McCrae, J., & Fellbaum, C. (2016). CILI: the Collaborative Interlingual Index. In *Proceedings of the 8th Global WordNet Conference*, 50-57.

Bormann, C. & Hoffman, P. (2020). Concise Binary Object Representation (CBOR). RFC 8949 / STD 94.

Buffer (2026). The State of Social Media Engagement in 2026: 52M+ Posts Analyzed. https://buffer.com/resources/state-of-social-media-engagement-2026/

Bush, V. (1945). As We May Think. *The Atlantic Monthly*, July 1945.

Chambers, J. (2025). *Keymaster: Open-Hardware Personal Vault and Key-Storage Device* [Project repository]. https://github.com/joshualibrarian/keymaster

Chambers, J. (2026). *Common Graph: Reference Implementation of the Frame Substrate* [Software repository]. https://github.com/joshualibrarian/common-graph

Clarke, I., Sandberg, O., Wiley, B., & Hong, T. W. (2001). Freenet: A Distributed Anonymous Information Storage and Retrieval System. In *Designing Privacy Enhancing Technologies*, Springer, 46-66.

Cohen, B. (2003). Incentives Build Robustness in BitTorrent. In *Workshop on Economics of Peer-to-Peer Systems*.

Fillmore, C. J. (1968). The Case for Case. In Bach, E. & Harms, R. T. (Eds.), *Universals in Linguistic Theory*, 1-88. Holt, Rinehart & Winston.

Fillmore, C. J. (1982). Frame Semantics. In *Linguistics in the Morning Calm*, 111-137. Seoul: Hanshin Publishing.

Flajolet, P., Fusy, É., Gandouet, O., & Meunier, F. (2007). HyperLogLog: the analysis of a near-optimal cardinality estimation algorithm. In *Proceedings of the 2007 Conference on Analysis of Algorithms (AofA)*, DMTCS.

Golbeck, J. (2005). *Computing and Applying Trust in Web-based Social Networks*. Ph.D. dissertation, University of Maryland, College Park.

Goldberg, D., Nichols, D., Oki, B. M., & Terry, D. (1992). Using collaborative filtering to weave an information tapestry. *Communications of the ACM*, 35(12), 61-70.

Gruber, T. R. (1995). Toward Principles for the Design of Ontologies Used for Knowledge Sharing. *International Journal of Human-Computer Studies*, 43(5-6), 907-928.

ISO 24617-4:2014. *Language Resource Management — Semantic Annotation Framework (SemAF) — Part 4: Semantic Roles (SemAF-SR)*. International Organization for Standardization, Geneva.

Kipper-Schuler, K. (2005). *VerbNet: A Broad-Coverage, Comprehensive Verb Lexicon*. Ph.D. dissertation, University of Pennsylvania.

Kleppmann, M., Wiggins, A., van Hardenberg, P., & McGranaghan, M. (2019). Local-First Software: You Own Your Data, in spite of the Cloud. In *Onward! 2019*, ACM.

Lenat, D. B. (1995). CYC: A Large-Scale Investment in Knowledge Infrastructure. *Communications of the ACM*, 38(11), 33-38.

Merkle, R. C. (1979). *Secrecy, Authentication, and Public Key Systems*. Ph.D. dissertation, Stanford University.

Milgram, S. (1967). The Small-World Problem. *Psychology Today*, 1(1), 61-67.

Miller, G. A. et al. (1993). *Introduction to WordNet: An On-line Lexical Database*. Princeton University.

Nelson, T. (1974). *Computer Lib / Dream Machines*. Self-published.

Pike, R. et al. (1995). Plan 9 from Bell Labs. *Computing Systems*, 8(3), 221-254.

Resnick, P., Iacovou, N., Suchak, M., Bergstrom, P., & Riedl, J. (1994). GroupLens: An open architecture for collaborative filtering of netnews. In *Proceedings of CSCW '94*, 175-186.

Shamir, A. (1979). How to Share a Secret. *Communications of the ACM*, 22(11), 612-613.

Shapiro, M. et al. (2011). Conflict-free Replicated Data Types. In *SSS 2011*, Springer.

Smith, D. A., Kay, A., Raab, A., & Reed, D. P. (2003). Croquet — A Collaboration System Architecture. In *Proceedings of C5*, IEEE.

Szabo, N. (1997). Formalizing and Securing Relationships on Public Networks. *First Monday*, 2(9).

Tarr, D., Lavoie, E., Meyer, A., & Tschudin, C. (2019). Secure Scuttlebutt: An Identity-Centric Protocol for Subjective and Decentralized Applications. In *ACM ICN '19*.

Viterbi, A. J. (1967). Error Bounds for Convolutional Codes and an Asymptotically Optimum Decoding Algorithm. *IEEE Transactions on Information Theory*, 13(2), 260-269.

Watts, D. J., & Strogatz, S. H. (1998). Collective Dynamics of 'Small-World' Networks. *Nature*, 393(6684), 440-442.

Youn, H. et al. (2016). On the Universal Structure of Human Lexical Semantics. *PNAS*, 113(7), 1766-1771.

Zimmermann, P. R. (1995). *The Official PGP User's Guide*. MIT Press.